

10_context&memory 大纲

1、概述	1
1.1 为什么需要记忆(Memory)	1
情况1：不同messages	1
情况2：同一个messages列表	2
1.2 如何解决记忆问题	3
1.2.1 上下文工程	3
1.2.2 上下文类型及相关的API	3
1.3 LangChain的记忆	3
1.3.1 记忆的分类	4
1.3.2 记忆的管理	4
2、短期记忆	4
2.1 基于内存的持久化器	5
2.1.1 举例1：没有记忆	5
2.1.2 举例2：拥有记忆	5
步骤1：第一二轮对话	5
步骤2：第三轮对话	8
步骤3：更新线程ID	12
2.1.3 关键步骤说明	13
场景1：多用户聊天	13
场景2：同一用户的不同任务	14
2.1.4 工作原理	14
2.1.5 常见问题	15
1、为什么 Agent 不记得？	15
2、InMemorySaver 会丢失数据吗？	15
3、内存会无限增长吗？	15
4、如何清空某个会话的历史？	16
2.2 基于外部存储介质的持久化器	16
2.2.1 数据库环境准备	16
2.2.2 代码实现	16
2.2.3 查看持久化数据	20
需求1：查看所有数据库	20
需求2：查看所有schema	20
需求3：查看当前所处的schema	21
需求4：查看当前schema下的所有表	21
2.3 对比两种方式	21
举例1：基于内存存储	21
举例2：基于外部存储器存储	23
2.4 记忆治理策略（上下文管理）	27
2.4.1 消息裁剪	28
2.4.2 消息删除	31
2.4.3 摘要	35

2.4.4 自定义过滤策略	39
2.5 了解：state的理解	44
3、长期记忆	47
3.1 基本理解	47
3.1.1 什么是长期记忆	47
3.1.2 类型划分	48
3.1.3 存储架构	49
3.2 基础API的使用	50
3.2.1 put()/get()：写入/读取API	50
① put()源码剖析	50
② get()源码剖析	51
③ 举例1：基于InMemoryStore	51
举例1：添加	51
举例2：更新	52
④ 举例2：基于PostgresStore	52
举例1：添加	52
举例2：更新	52
3.2.3 search()：检索API	53
① 源码剖析	53
② 举例1：按照namespace前缀搜索	53
③ 举例2：按照filter过滤	55
④ 举例3：按照语义搜索	56
举例1：自定义嵌入函数	56
举例2：使用嵌入模型	58
3.3 在Agent运行图中访问长期记忆	60
3.3.1 在工具中访问长期记忆	60
① 基于InMemoryStore	60
② 基于PostgresStore	62
3.3.2 在中间件中访问长期记忆	65
① Node-style hooks中访问	65
② Wrap-style hooks中访问	65
1. wrap_model_call	65
2. wrap_tool_call	66
3.4 何时写入记忆	67
1. 在主流程里写（hot path）	67
2. 在后台写（background）	67
4、课后阅读：Static runtime Context	67
4.1 中间件中访问	68
4.1.1 Node-style hooks	68
4.1.2 Wrap-style hooks	71
① wrap_model_call	71
② wrap_tool_call	76
4.1.3 特殊的便捷中间件dynamic_prompt	79
4.2 工具中访问	83

1、概述

1.1 为什么需要记忆(Memory)

记忆是一种记住之前互动信息的系统。随着Agent处理涉及大量用户交互的复杂任务，记忆变得至关重要！

大多数的大模型应用程序都会有一个会话接口，允许我们进行多轮的对话，并有一定的上下文记忆能力。

但实际上，大模型本身是“无状态”的，不会记忆任何上下文的。即每次调用 `agent.invoke()` 都是全新的开始，不记得之前的对话。

```
In [1]: from langchain.chat_models import init_chat_model
        from dotenv import load_dotenv
        import os

        # 从 .env 文件中加载环境变量
        load_dotenv(override=True)
        model = init_chat_model(
            model="qwen3.7-flash",
            model_provider="openai",
            api_key=os.getenv("DASHSCOPE_API_KEY"),
            base_url=os.getenv("DASHSCOPE_BASE_URL")
        )
```

情况1：不同messages

```
In [2]: from langchain.agents import create_agent
        from langchain_core.messages import HumanMessage, AIMessage

        messages1 = [
            HumanMessage("你好，我叫小明"),
            AIMessage("很高兴认识你，小明"),
            HumanMessage("你用一句话介绍下你自己"),
        ]

        agent = create_agent(
            model=model,
            tools=[],
        )

        response1 = agent.invoke({"messages": messages1})
        for msg in response1["messages"]:
            msg.pretty_print()
```

```
c:\Users\huan.zheng\AppData\Local\miniconda3\envs\langchain1\Lib\site-packages\langgraph
\checkpoint\base\__init__.py:17: LangChainPendingDeprecationWarning: The default value of `a
llowed_objects` will change in a future version. Pass an explicit value (e.g., allowed_objects='me
ssages' or allowed_objects='core') to suppress this warning.
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
```

===== Human Message =====

你好，我叫小明

===== Ai Message =====

很高兴认识你，小明

===== Human Message =====

你用一句话介绍下你自己

===== Ai Message =====

我是一个人工智能助手，致力于用清晰、准确且温暖的方式为你的学习、工作与日常需求提供随时随地的帮助。

```
In [3]: messages2 = [HumanMessage("我叫什么名字？")]
response2 = agent.invoke({"messages": messages2})
for msg in response2["messages"]:
    msg.pretty_print()
```

===== Human Message =====

我叫什么名字？

===== Ai Message =====

我不知道你的名字哦，因为我们刚刚开始对话，你还没有告诉我。如果你愿意的话，可以直接告诉我你的称呼，这样我以后就能用名字亲切地称呼你啦！😊

情况2：同一个messages列表

```
In [4]: messages3 = [
    HumanMessage("你好，我叫小明"),
    AIMessage("很高兴认识你，小明"),
    HumanMessage("你用一句话介绍下你自己"),
    AIMessage("我是一个由 OpenAI 训练的人工智能助手，可以帮你回答问题、写作和解决各种任务。"),
    HumanMessage("我叫什么名字？")
]

response3 = agent.invoke({"messages": messages3})

for msg in response3["messages"]:
    msg.pretty_print()
```

```
===== Human Message =====

你好，我叫小明

===== Ai Message =====

很高兴认识你，小明

===== Human Message =====

你用一句话介绍下你自己

===== Ai Message =====

我是一个由 OpenAI 训练的人工智能助手，可以帮你回答问题、写作和解决各种任务。

===== Human Message =====

我叫什么名字？

===== Ai Message =====

你叫小明。
```

1.2 如何解决记忆问题

1.2.1 上下文工程

实现这个记忆功能，就需要额外的模块去保存我们和模型对话的上下文信息，然后在下一次请求时，把历史信息都输入给模型，让模型输出结果。在 LangChain 中，记忆(Memory) 就是专门负责“存储历史交互信息”的组件，核心作用是「保存上下文」和「提供上下文」，让 LLM在每次响应时，都能“看到”之前的对话内容。

上下文工程(Context Engineering) 负责“合理组织”这些记忆和任务信息，让LLM的响应更连贯、更贴合需求。这也是Agent能实现复杂多轮交互的核心基础。

1.2.2 上下文类型及相关的API

LangChain的上下文工程是基于Agent讨论的，而上下文工程是构建在LangGraph之上的。LangGraph 提供了三种管理上下文的方法，这些方法结合了可变性和生命周期维度：

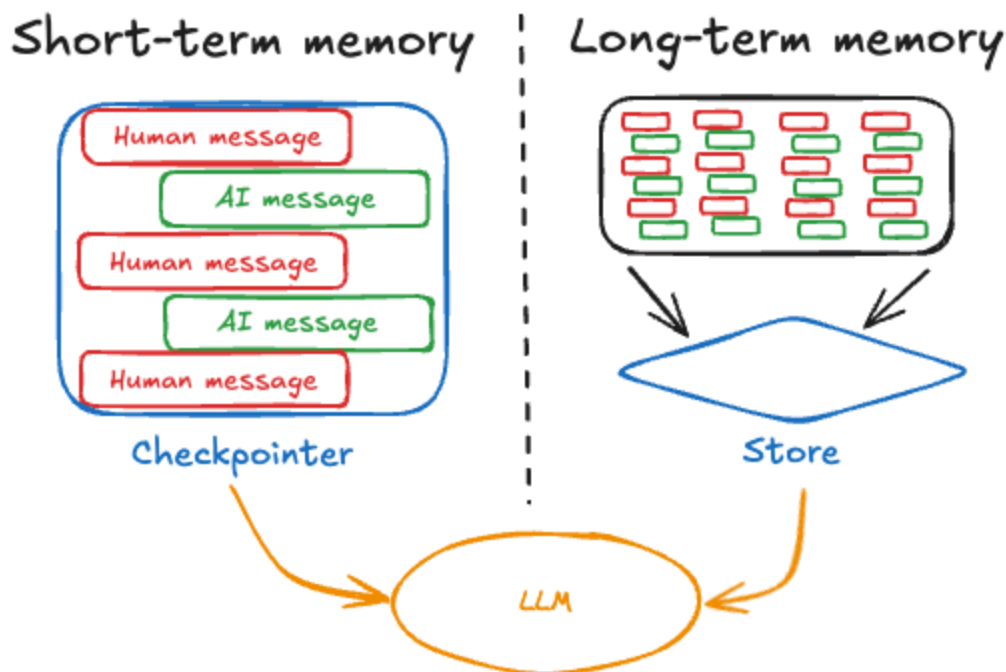
上下文类型	描述	可变性	生命周期	访问方法
动态运行时上下文	在单次运行中会演变的可变数据	动态	单次运行	LangGraph state对象
动态跨会话上下文	在对话间共享的持久数据。比如用户偏好、历史洞察、知识条目	动态	跨对话	LangGraph store对象
静态运行时上下文	在启动时传入的用户元数据、工具、数据库连接	静态	单次运行	LangGraph context对象

1.3 LangChain的记忆

1.3.1 记忆的分类

官方说明：<https://docs.langchain.com/oss/python/concepts/memory> 记忆分为短期记忆和长期记忆，对应不同的使用场景：

- 短期记忆（Short-term memory、会话级记忆、thread-scoped memory）：作用范围是单个对话线程（Thread）内，一旦开启新对话（更换 thread_id），记忆即消失。
- 长期记忆（Long-term memory，跨会话级记忆）：在会话间存储用户特定或应用级数据，并在会话线程间共享。它可以随时在任何线程中被调用。记忆的范围是任意自定义命名空间，而不仅仅是单一线程 ID。



1.3.2 记忆的管理

在LangChain v0.x版本中，通过专用的xxxMemory类管理记忆。

在LangChain v1.x版本中，Agent是构建在LangGraph图结构之上的，通过上文提到的state和store构建记忆系统。使用更简单、功能更统一。

- state：短期记忆对象，以会话为单位组织，包含当前会话的所有消息记录以及自定义信息。
- store：长期记忆对象，跨会话持久化的数据，通常需要结合向量数据库或外部存储实现。

2、短期记忆

LangChain1.x 的短期记忆是三者的组合：

State（会话内部状态） + Checkpointer（持久化机制） + Thread ID（会话作用域）

- State：默认存储历史消息列表messages，通过State管理历史消息
- Checkpointer：负责将State作为检查点持久化保存，检查点是某个时刻的State快照
- Thread ID：用于唯一标识State，LangChain运行时会根据thread_id读写State快照

这就像玩RPG游戏时的“自动存档”：你不需要手动保存，系统在关键节点自动记录，下次进入游戏随时可以从上次的存档点继续。

2.1 基于内存的持久化器

这是最便捷的使用方式，适合快速测试或调试。

2.1.1 举例1：没有记忆

```
In [5]: from langchain.agents import create_agent
from langchain.messages import HumanMessage

agent = create_agent(
    model=model,
    tools=[]
)

print("\n第一轮对话：")
response1 = agent.invoke({
    "messages": [HumanMessage("我叫张三")]
})
print(f"Agent: {response1['messages'][-1].content}")

print("\n第二轮对话：")
response2 = agent.invoke({
    "messages": [HumanMessage("我叫什么？")]
})
print(f"Agent: {response2['messages'][-1].content}")
```

第一轮对话：

Agent: 你好，张三！很高兴认识你。请问今天有什么我可以帮你的吗？

第二轮对话：

Agent: 我不知道你的名字哦。作为 AI 助手，我无法获取或存储你的个人信息。如果你愿意告诉我，我很乐意在之后的对话中这样称呼你！😊

2.1.2 举例2：拥有记忆

步骤1：第一二轮对话

```
In [6]: from langgraph.checkpoint.memory import InMemorySaver
checkpointer = InMemorySaver()
```

```

# 1. 创建 Agent 时添加 checkpointer
agent = create_agent(
    model=model,
    checkpointer=checkpointer # 添加内存管理
)

# 2. 调用时指定 thread_id
config = {
    "configurable": {
        "thread_id": "1"
    }
}

print("\n第一轮对话：")
response1 = agent.invoke({
    "messages": [HumanMessage("我叫张三")],
    config=config # 传入 config
})
print(f"Agent: {response1['messages'][-1].content}")

print("\n第二轮对话：")
response2 = agent.invoke({
    "messages": [HumanMessage("我叫什么？")],
    config=config # 使用相同的 thread_id
})
print(f"Agent: {response2['messages'][-1].content}")

```

第一轮对话：

Agent: 你好，张三！很高兴认识你。今天有什么我可以帮你的吗？

第二轮对话：

Agent: 你叫**张三**。刚才你亲口告诉我的，我已经记住啦！今天有什么我可以帮你的吗？

说明：你只需传入 checkpointer 和 config，Agent 就能自然具备连续对话能力。

```

In [7]: from rich import print as rprint

latest_state = agent.get_state(config)
rprint(latest_state)

```



```

StateSnapshot(
  values={
    'messages': [
      HumanMessage(
        content='我叫张三',
        additional_kwargs={},
        response_metadata={},
        id='1eb1297f-8f9a-4d3c-b3ab-ef431054ccea'
      ),
      AIMessage(
        content='你好，张三！很高兴认识你。今天有什么我可以帮你的吗？',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 387,
            'prompt_tokens': 12,
            'total_tokens': 399,
            'completion_tokens_details': {
              'accepted_prediction_tokens': None,
              'audio_tokens': None,
              'reasoning_tokens': 367,
              'rejected_prediction_tokens': None,
              'text_tokens': 387
            },
            'prompt_tokens_details': {'audio_tokens': None}
          },
          'model_provider': 'openai',
          'model_name': 'qwen3.7-flash',
          'system_fingerprint': None,
          'id': 'chatcmpl-2bcf785b-dc87-9e96-83ef-396f75fe64',
          'finish_reason': 'stop',
          'logprobs': None
        },
        id='lc_run--01a0192d-c6a7-7da1-b442-37b693e72d8e-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 12,
          'output_tokens': 387,
          'total_tokens': 399,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {'reasoning': 367}
        }
      ),
      HumanMessage(
        content='我叫什么？',
        additional_kwargs={},
        response_metadata={},
        id='64c52567-1109-48ce-bb15-e3963934a5d4'
      ),
      AIMessage(
        content='你叫**张三**。刚才你亲口告诉我的，我已经记住啦！今天有',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 337,
            'prompt_tokens': 40,
            'total_tokens': 377,
            'completion_tokens_details': {

```

```

        'accepted_prediction_tokens': None,
        'audio_tokens': None,
        'reasoning_tokens': 309,
        'rejected_prediction_tokens': None,
        'text_tokens': 337
    },
    'prompt_tokens_details': None
},
'model_provider': 'openai',
'model_name': 'qwen3.7-flash',
'system_fingerprint': None,
'id': 'chatcmpl-b97c9224-5d06-9fd6-987b-a81c4f4ca9',
'finish_reason': 'stop',
'logprobs': None
},
id='lc_run--01a0192d-d202-7713-85c3-cdab781e6b14-0',
tool_calls=[],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 40,
    'output_tokens': 337,
    'total_tokens': 377,
    'input_token_details': {},
    'output_token_details': {'reasoning': 309}
}
    )
]
},
next=(),
config={
    'configurable': {
        'thread_id': '1',
        'checkpoint_ns': '',
        'checkpoint_id': '1f19ba9a-2fc9-67bd-8004-0fd2fd4cf540'
    }
},
metadata={'source': 'loop', 'step': 4, 'parents': {}},
created_at='2026-08-19T08:40:31.143724+00:00',
parent_config={
    'configurable': {
        'thread_id': '1',
        'checkpoint_ns': '',
        'checkpoint_id': '1f19ba9a-15c7-6c62-8003-deda076764a5'
    }
},
tasks=(),
interrupts=()
)

```

步骤2：第三轮对话

当我们再次进行对话时，直接带入线程ID，即可带入此前对话记忆：

```

In [8]: print("\n第三轮对话：")
response3 = agent.invoke({
    "messages": [HumanMessage("我刚才问了什么问题?")],
    config=config # 使用相同的 thread_id

```

```
)  
print(f"Agent: {response3['messages'][-1].content}")
```

第三轮对话：

Agent: 你刚才问的是：***“我叫什么？”***

如果还有其他问题或需要帮忙，随时告诉我！

In [9]:

```
from rich import print as rprint  
latest_state = agent.get_state(config)  
rprint(latest_state)
```

```

StateSnapshot(
  values={
    'messages': [
      HumanMessage(
        content='我叫张三',
        additional_kwargs={},
        response_metadata={},
        id='1eb1297f-8f9a-4d3c-b3ab-ef431054ccea'
      ),
      AIMessage(
        content='你好，张三！很高兴认识你。今天有什么我可以帮你的吗？',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 387,
            'prompt_tokens': 12,
            'total_tokens': 399,
            'completion_tokens_details': {
              'accepted_prediction_tokens': None,
              'audio_tokens': None,
              'reasoning_tokens': 367,
              'rejected_prediction_tokens': None,
              'text_tokens': 387
            },
            'prompt_tokens_details': {'audio_tokens': None}
          },
          'model_provider': 'openai',
          'model_name': 'qwen3.7-flash',
          'system_fingerprint': None,
          'id': 'chatcmpl-2bcf785b-dc87-9e96-83ef-396f75fe64',
          'finish_reason': 'stop',
          'logprobs': None
        },
        id='lc_run--01a0192d-c6a7-7da1-b442-37b693e72d8e-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 12,
          'output_tokens': 387,
          'total_tokens': 399,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {'reasoning': 367}
        }
      ),
      HumanMessage(
        content='我叫什么？',
        additional_kwargs={},
        response_metadata={},
        id='64c52567-1109-48ce-bb15-e3963934a5d4'
      ),
      AIMessage(
        content='你叫**张三**。刚才你亲口告诉我的，我已经记住啦！今天有',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 337,
            'prompt_tokens': 40,
            'total_tokens': 377,
            'completion_tokens_details': {

```

```

        'accepted_prediction_tokens': None,
        'audio_tokens': None,
        'reasoning_tokens': 309,
        'rejected_prediction_tokens': None,
        'text_tokens': 337
    },
    'prompt_tokens_details': None
},
'model_provider': 'openai',
'model_name': 'qwen3.7-flash',
'system_fingerprint': None,
'id': 'chatcmpl-b97c9224-5d06-9fd6-987b-a81c4f4ca9',
'finish_reason': 'stop',
'logprobs': None
},
id='lc_run--01a0192d-d202-7713-85c3-cdab781e6b14-0',
tool_calls=[],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 40,
    'output_tokens': 337,
    'total_tokens': 377,
    'input_token_details': {},
    'output_token_details': {'reasoning': 309}
}
),
HumanMessage(
    content='我刚才问了什么问题？',
    additional_kwargs={},
    response_metadata={},
    id='ee88240a-dd32-42eb-9f88-608eab0eb756'
),
AIMessage(
    content='你刚才问的是：***我叫什么？*** \n\n如果还有其他问题可
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 366,
            'prompt_tokens': 78,
            'total_tokens': 444,
            'completion_tokens_details': {
                'accepted_prediction_tokens': None,
                'audio_tokens': None,
                'reasoning_tokens': 340,
                'rejected_prediction_tokens': None,
                'text_tokens': 366
            },
        },
        'prompt_tokens_details': None
    },
    'model_provider': 'openai',
    'model_name': 'qwen3.7-flash',
    'system_fingerprint': None,
    'id': 'chatcmpl-206291a1-e719-96f4-b062-d69bf02421',
    'finish_reason': 'stop',
    'logprobs': None
},
id='lc_run--01a01932-60ac-71c3-af07-599819e6a9bf-0',
tool_calls=[],
invalid_tool_calls=[],

```

```

        usage_metadata={
            'input_tokens': 78,
            'output_tokens': 366,
            'total_tokens': 444,
            'input_token_details': {},
            'output_token_details': {'reasoning': 340}
        }
    ],
    next=(),
    config={
        'configurable': {
            'thread_id': '1',
            'checkpoint_ns': '',
            'checkpoint_id': '1f19baa5-54e5-6ff0-8007-e3b58436a575'
        }
    },
    metadata={'source': 'loop', 'step': 7, 'parents': {}},
    created_at='2026-08-19T08:45:30.314136+00:00',
    parent_config={
        'configurable': {
            'thread_id': '1',
            'checkpoint_ns': '',
            'checkpoint_id': '1f19baa5-3612-6092-8006-3a6bedb4b4f7'
        }
    },
    tasks=(),
    interrupts=()
)

```

说明：第三轮对话中的“我刚才问了什么问题？”就能在第二轮存储的消息历史中找到答案。所有消息历史都会自动追加到 `AgentState` 的 `messages` 字段中，无需手动维护。

步骤3：更新线程ID

而如果更新线程ID，则会重新开启对话：

```

In [10]: config2 = {
    "configurable": {
        "thread_id": "2"
    }
}
response4 = agent.invoke(
    {"messages": [HumanMessage("你还记得我叫什么名字么？")]},
    config=config2
)
print(response4['messages'][-1].content)

```

很抱歉，我还不知道你的名字呢。作为人工智能，我无法自动获取或记住你的个人真实身份。

不过，如果你愿意告诉我，我很乐意记住它并在以后的交流中这样称呼你。你可以告诉我你叫什么吗？



说明：thread_id 隔离不同会话空间。

2.1.3 关键步骤说明

第1步：初始化记忆引擎： `checkpointer = InMemorySaver()` ——创建一个内存级的记忆存储。

注意：`InMemorySaver`内存中保存，进程结束就丢失数据，适合测试。生产环境可换成数据库持久化的 `SqliteSaver`、`PostgresSaver` 等

第2步：绑定 Agent：在 `create_agent` 时传入 `checkpointer`，让 Agent 具备状态存储能力。

第3步：设定会话 ID：通过 `config = {"configurable": {"thread_id": "1"}}` 为每次调用指定线程标识。

同一个 `thread_id` 共享记忆，不同 `thread_id` 完全隔离。

```
In [ ]: # 会话 1
        config1 = {"configurable": {"thread_id": "1"}}
        agent.invoke(..., config=config1)

        # 会话 2
        config2 = {"configurable": {"thread_id": "2"}}
        agent.invoke(..., config=config2)

        # 两个会话完全独立
```

`thread_id` 是记忆管理的核心开关：在会话2里询问会话1的会话信息，Agent 会表示不知道——因为双方记忆空间完全隔离。

生产环境中：

场景1：多用户聊天

不同 `thread_id` = 不同会话，Agent 能正确记住每个会话的内容。

```
In [ ]: agent = create_agent(
        model=model,
        tools=[],
        checkpointer=InMemorySaver()
    )

    # 用户 Alice
    config_alice = {"configurable": {"thread_id": "user_alice"}}
    agent.invoke({"messages": [...]}, config_alice)
    ...

    # 用户 Bob
    config_bob = {"configurable": {"thread_id": "user_bob"}}
    agent.invoke({"messages": [...]}, config_bob)
    ...

    # 两个会话完全独立
```

场景2：同一用户的不同任务

```
In [ ]: # 任务 1：写代码
config_task1 = {"configurable": {"thread_id": "task_coding"}}
agent.invoke({"messages": [...]}, config_task1)
...

# 任务 2：写文档
config_task2 = {"configurable": {"thread_id": "task_docs"}}
agent.invoke({"messages": [...]}, config_task2)
...
```

2.1.4 工作原理

内存保存了什么？

```
In [ ]: agent.invoke({"messages": [{"role": "user", "content": "你好"}]}, config)
# InMemorySaver 保存：
# {
#     "thread_id": "xxx",
#     "messages": [
#         HumanMessage("你好"),
#         AIMessage("你好！有什么可以帮助你吗？")
#     ]
# }
agent.invoke({"messages": [{"role": "user", "content": "天气"}]}, config)
# InMemorySaver 更新：
# {
#     "thread_id": "xxx",
#     "messages": [
#         HumanMessage("你好"),
#         AIMessage("你好！有什么可以帮助你吗？"),
#         HumanMessage("天气"),
#         AIMessage("...")
#     ]
# }
```

自动追加历史

```
In [ ]: # 你只需要传新消息
agent.invoke(
    {"messages": [{"role": "user", "content": "新问题"}]},
    config
)
```

此时，checkpointer 自动：

- ① 读取之前的历史
- ② 追加新消息
- ③ 调用模型（传入完整历史）

④ 保存新的历史

说明：checkpointer 会自动管理历史

2.1.5 常见问题

1、为什么 Agent 不记得？

检查：

- 是否添加了 `checkpointer=InMemorySaver()` ？
- 是否传入了 `config` 参数？
- 两次调用的 `thread_id` 是否相同？

```
In [ ]: # ❌ 错误：没有 checkpointer
agent = create_agent(model=model, tools=[])
agent.invoke({...}) # 不会记住

# ❌ 错误：没有 config
agent = create_agent(model=model, tools=[], checkpointer=InMemorySaver())
agent.invoke({...}) # 不会记住

# ❌ 错误：thread_id 不同
agent.invoke(..., config={"configurable": {"thread_id": "1"}})
agent.invoke(..., config={"configurable": {"thread_id": "2"}}) # 不同会话

# ✅ 正确
agent = create_agent(model=model, tools=[], checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "1"}}
agent.invoke(..., config)
agent.invoke(..., config) # 记得！
```

2、InMemorySaver 会丢失数据吗？

会！InMemorySaver 只保存在内存中：

- ✅ 同一进程内有效（不支持跨进程共享）
- ❌ 程序重启后丢失（或进程重启后丢失）
- ❌ 不同进程无法共享

解决方案：持久化（SQLite、PostgreSQL）

3、内存会无限增长吗？

会！默认情况下，InMemorySaver 会保存所有消息。

问题：

- 消息越来越多（无限增长，需要管理上下文）
- token消耗增加，甚至会超过模型的 token 限制
- 响应速度变慢、成本增加

解决方案：上下文管理（修剪、摘要）

4、如何清空某个会话的历史？

目前 InMemorySaver 没有提供删除 API。

临时方案：

- 使用新的 thread_id
- 或重新创建 Agent

2.2 基于外部存储介质的持久化器

如果将状态检查点(checkpointer) 保存在内存， 进程结束则状态丢失， 生产环境不可接受。因此， 生产环境要用持久化的外部存储介质， 如PostgreSQL。 LangGraph提供的 checkpointer后端列表如下

<https://docs.langchain.com/oss/python/langgraph/persistence#checkpointer-libraries>

此处选择PostgreSQL作为持久化器。

2.2.1 数据库环境准备

在此之前，先准备好PostgreSQL环境，此处云服务器的Ubuntu系统安装PostgreSQL。大家需要根据自己云服务器位置，更改URL中的IP即可。

具体云服务器安装和PostgreSQL的安装，见《Linux云服务器安装与PostgreSQL安装》

URL：

```
postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
sslmode=disable
```

说明：

- 1、上述URL中的用户名、密码、IP地址，需要根据自己的情况替换。
- 2、要在LangChain中对接PostgreSQL，还需要额外的依赖，比如：

```
In [ ]: # pip install langgraph-checkpoint-postgres
```

其实我们在课程开始的requirements.txt中已经提供了相关依赖，此处不必也不能重新安装，否则版本可能冲突。

2.2.2 代码实现

```
In [5]: from langchain.chat_models import init_chat_model
        from dotenv import load_dotenv
        import os

        # 从.env文件中加载环境变量
        load_dotenv(override=True)
```

```

model = init_chat_model(
    model="qwen3.7-flash",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)

```

```

In [10]: from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.postgres import PostgresSaver

DB_URL = "postgresql://langchain_user:abcd1234@119.45.243.141:5432/langchain_db?sslmode=

with PostgresSaver.from_conn_string(DB_URL) as checkpointer:
    # 初始化PostgreSQL数据库
    checkpointer.setup()

    agent = create_agent(
        model=model,
        checkpointer=checkpointer
    )

    config = {"configurable": {"thread_id": "1"}}
    response1 = agent.invoke(
        {"messages": [HumanMessage("你好，我是老王")]},
        config=config
    )

    print("=" * 30, "-> 第一次调用 <=", "=" * 30)
    for msg in response1["messages"]:
        msg.pretty_print()

    response2 = agent.invoke(
        {"messages": [HumanMessage("你好，我是谁？")]},
        config=config
    )

    print("=" * 30, "-> 第二次调用 <=", "=" * 30)
    for msg in response2["messages"]:
        msg.pretty_print()

```

===== -> 第一次调用 <- =====
===== Human Message =====

你好，我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？无论是聊天、解答问题还是处理事务，随时告诉我～

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据你刚才的自我介绍，你是**老王**呀！😊

不过得坦诚跟你说，作为 AI 我并没有长期记忆，也不会在我们聊天之外了解你的身份。我只知道你在当前的对话里告诉了我这个名字。如果你想让我以后用特定的称呼、记住某些偏好，也可以随时告诉我，我会尽量配合。

今天找我是想随便聊聊，还是有什么具体事情需要帮忙？

===== Human Message =====

你好，我是老王

===== Ai Message =====

你好呀，老王！😊 看来咱们的对话又回到开头啦～没错，在这个聊天里，你就是你亲口告诉我的“老王”。今天是想随便唠唠，还是有什么具体事情、问题需要我搭把手？随时说，我都在！

===== Human Message =====

你好，我是谁？

===== Ai Message =====

还是你呀～根据咱们刚才的对话，你亲口告诉我你是**老王**。在我这儿，你就是此刻和我聊天、名字叫“老王”的朋友。

如果这是你对自己身份的哲学拷问，那答案可能是：你是独一无二的你；如果只是确认一下我的记忆状态，那我也能稳稳接住——我知道你在本次聊天里告诉了我名字，但如果你以后换了称呼或补充新设定，我也会随时更新。

老王，今天想随便聊聊，还是有什么具体问题需要我搭把手？

===== Human Message =====

你好，我是老王

===== Ai Message =====

你好呀，老王！👋
看到你再次打招呼啦～这次咱们干脆直接进正题：今天想随便聊聊、解决某个具体问题，还是单纯想看看我的反应？随时吩咐，我随时在线。😊

===== -> 第二次调用 <- =====
===== Human Message =====

你好，我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？无论是聊天、解答问题还是处理事务，随时告诉我～

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据你刚才的自我介绍，你是**老王**呀！😊

不过得坦诚跟你说，作为 AI 我并没有长期记忆，也不会在我们聊天之外了解你的身份。我只知道你在当前的对话里告诉了我这个名字。如果你想让我以后用特定的称呼、记住某些偏好，也可以随时告诉我，我会尽量配合。

今天找我是想随便聊聊，还是有什么具体事情需要帮忙？

===== Human Message =====

你好，我是老王

===== Ai Message =====

你好呀，老王！😊 看来咱们的对话又回到开头啦～没错，在这个聊天里，你就是你亲口告诉我的“老王”。今天是想随便唠唠，还是有什么具体事情、问题需要我搭把手？随时说，我都在！

===== Human Message =====

你好，我是谁？

===== Ai Message =====

还是你呀～根据咱们刚才的对话，你亲口告诉我你是**老王**。在我这儿，你就是此刻和我聊天、名字叫“老王”的朋友。

如果这是你对自己身份的哲学拷问，那答案可能是：你是独一无二的你；如果只是确认一下我的记忆状态，那我也能稳稳接住——我知道你在本次聊天里告诉了我名字，但如果你以后换了称呼或补充新设定，我也会随时更新。

老王，今天想随便聊聊，还是有什么具体问题需要我搭把手？

===== Human Message =====

你好，我是老王

===== Ai Message =====

你好呀，老王！👋

看到你再次打招呼啦～这次咱们干脆直接进正题：今天想随便聊聊、解决某个具体问题，还是单纯想看看我的反应？随时吩咐，我随时在线。😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

还是你呀——在这段对话里，你亲口告诉我了，你是**老王**。👋

看你来回切换“报名字”和“灵魂发问”，我猜你要么是在测我的反应，要么就是想轻松唠两句？都没关系，老王今天想随便聊聊、解决实际问题，还是单纯想看看我会怎么接招？随时说，我在呢～😊

setup() 用于初始化PostgreSQL数据库，**首次运行会创建必要的表，重复执行不会重新建表**，底层逻辑是Create IF Not Exists，相关源码如下

```
In [ ]: def setup(self) -> None:
        """Set up the checkpoint database asynchronously.
```

This method creates the necessary tables in the Postgres database if they don't already exist and runs database migrations. It MUST be called directly by the user the first time checkpointer is used.

```
""""  
...  
"""
```

2.2.3 查看持久化数据

查看PostgreSQL数据库，可以通过命令行或图形化工具查看

```
In [ ]: ubuntu@VM-0-6-ubuntu:~$ psql "postgresql://langchain_user:abcd1234@localhost:5432/langcha
```

PostgreSQL的存储结构是Database -> Schema -> Table

- Database：数据库
- Schema：分区
- Table：表

需求1：查看所有数据库

输入指令：\l

```
ubuntu@VM-0-12-ubuntu:/home/ubuntu [Shell] + Agent  
ubuntu@VM-0-12-ubuntu:~$ psql "postgresql://langchain_user:abcd1234@localhost:5432/langchain_db?sslmode=disable"  
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))  
Type "help" for help.  
  
langchain_db=> \l  
  
          List of databases  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
 Name      | Owner      | Encoding | Locale Provider | Collate  | Ctype     | ICU Locale | ICU Rules | Access privileges  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
 langchain_db | langchain_user | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |             |           | =Tc/langchain_user  
               |               |          |                 |             |             |             |           | langchain_user=Ctc/langchain_user  
 postgres     | postgres     | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |             |           | =c/postgres  
               |               |          |                 |             |             |             |           | postgres=Ctc/postgres  
 template0    | postgres     | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |             |           | =c/postgres  
               |               |          |                 |             |             |             |           | postgres=Ctc/postgres  
 (4 rows)  
  
langchain_db=>
```

命令行前缀langgraph_db 标识了当前所在数据库

需求2：查看所有schema

输入指令:\dn

```
langchain_db=> \dn  
          List of schemas  
+-----+-----+  
 Name      | Owner  
+-----+-----+  
 public    | pg_database_owner  
 (1 row)
```

需求3：查看当前所处的schema

输入指令：select current_schema();

```
langchain_db=> select current_schema();
current_schema
-----
public
(1 row)
```

需求4：查看当前schema下的所有表

输入指令：\dt

```
langchain_db=> \dt

List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | checkpoint_blobs | table | langchain_user
public | checkpoint_migrations | table | langchain_user
public | checkpoint_writes | table | langchain_user
public | checkpoints | table | langchain_user
(4 rows)
```

这四张表都是setup() 函数初始化时创建的。

- checkpoints：这是主表，存每个 thread 在某个时刻的 checkpoint 快照。
- checkpoint_blobs：这张表专门存不适合直接内联进 checkpoints.checkpoint 的较复杂channel 值。
- checkpoint_writes：这张表存的是中间写入 / pending writes，不是最终完整 checkpoint。
- checkpoint_migrations：这张表不是业务数据表，而是迁移版本表。

2.3 对比两种方式

举例1：基于内存存储

```
In [13]: from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
```

```

model=model,
checkpointer=InMemorySaver()
)
config={"configurable":{"thread_id": "1"}}

print("=" * 30, "-> 第一次调用 <=", "=" * 30)
response1 = agent.invoke(
    {"messages": [HumanMessage("你好，我是谁？")]},
    config=config
)
for msg in response1["messages"]:
    msg.pretty_print()

print("=" * 30, "-> 第二次调用 <=", "=" * 30)
response2 = agent.invoke(
    {"messages": [HumanMessage("我是老王")]},
    config=config
)
for msg in response2["messages"]:
    msg.pretty_print()

print("=" * 30, "-> 第三次调用 <=", "=" * 30)
response3 = agent.invoke(
    {"messages": [HumanMessage("你好，我是谁？")]},
    config
)
for msg in response3["messages"]:
    msg.pretty_print()

```



```

===== -> 第一次调用 <- =====
===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能助手，我无法知道你的真实身份或个人信息，只能通过你主动提供的内容来了解你。如果你愿意告诉我你的名字、兴趣，或者具体想让我帮你做什么，我很乐意为你提供支持或陪你聊聊！😊

===== -> 第二次调用 <- =====
===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能助手，我无法知道你的真实身份或个人信息，只能通过你主动提供的内容来了解你。如果你愿意告诉我你的名字、兴趣，或者具体想让我帮你做什么，我很乐意为你提供支持或陪你聊聊！😊

===== Human Message =====

我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？不管是工作学习上的问题、生活里的小困扰，还是单纯想聊聊天，随时告诉我～😊

===== -> 第三次调用 <- =====
===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能助手，我无法知道你的真实身份或个人信息，只能通过你主动提供的内容来了解你。如果你愿意告诉我你的名字、兴趣，或者具体想让我帮你做什么，我很乐意为你提供支持或陪你聊聊！😊

===== Human Message =====

我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？不管是工作学习上的问题、生活里的小困扰，还是单纯想聊聊天，随时告诉我～😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据咱们刚才的聊天，你刚刚告诉我你叫**老王**。如果这次是重新开启的对话，或者你希望我换种方式称呼你，随时告诉我～今天有什么我可以帮你或陪你聊聊的吗？😊

```

可以发现，每次执行的输出完全相同，而我们并没有更改thread_id，之所以看不到上次运行的状态是因为每次运行创建新的Saver()，历史State被丢弃了。

举例2：基于外部存储器存储

```

In [14]: from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.postgres import PostgresSaver

DB_URL = "postgresql://langchain_user:abcd1234@119.45.243.141:5432/langchain_db?sslmode=
with PostgresSaver.from_conn_string(DB_URL) as checkpointer:
    # 初始化数据库
    checkpointer.setup()

    agent = create_agent(
        model=model,
        checkpointer=checkpointer
    )

    config = {"configurable":{"thread_id":"3"}}

    print("=" * 30, "-> 第一次调用 <-", "=" * 30)
    response1 = agent.invoke(
        {"messages": [HumanMessage("你好，我是谁？")]},
        config=config
    )
    for msg in response1["messages"]:
        msg.pretty_print()

    print("=" * 30, "-> 第二次调用 <-", "=" * 30)
    response2 = agent.invoke(
        {"messages": [HumanMessage("我是老王")]},
        config=config
    )
    for msg in response2["messages"]:
        msg.pretty_print()

    print("=" * 30, "-> 第三次调用 <-", "=" * 30)
    response3 = agent.invoke(
        {"messages": [HumanMessage("你好，我是谁？")]},
        config=config
    )
    for msg in response3["messages"]:
        msg.pretty_print()

```

===== -> 第一次调用 <- =====
===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能，我无法获取或记忆你的个人信息，也不知道你的真实身份。除非你主动告诉我，否则我会一直把你当作一位正在和我交流的用户。

如果你愿意，可以告诉我我希望我怎么称呼你，或者直接说说你今天需要什么帮助，我会尽力为你解答或提供支持！

===== Human Message =====

我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我会尽力配合！😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据我们刚才的对话，你亲口告诉过我你的名字***“老王”**呀！所以在我们的这段聊天里，你就是老王。😊

（顺便说明一下：我作为AI不会跨会话记忆个人信息，但只要你在这段对话中告诉我，我就会结合上下文和你自然交流～）

老王今天想聊点什么？或者有什么具体问题需要我帮忙？随时说，我在这儿！

===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好呀！在我们这段对话里，你刚刚亲口告诉我，你是**老王**。
虽然这个问题你又问了一遍，但我还是稳稳地记得：你就是老王～（是不是在悄悄测试我的记忆力呀？😊）

今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我都在！

===== -> 第二次调用 <- =====
===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能，我无法获取或记忆你的个人信息，也不知道你的真实身份。除非你主动告诉我，否则我会一直把你当作一位正在和我交流的用户。

如果你愿意，可以告诉我我希望我怎么称呼你，或者直接说说你今天需要什么帮助，我会尽力为你解答或提供支持！

===== Human Message =====

我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我会尽力配合！😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据我们刚才的对话，你亲口告诉过我你的名字**“老王”**呀！所以在我们的这段聊天里，你就是老王。😊

（顺便说明一下：我作为AI不会跨会话记忆个人信息，但只要你在这段对话中告诉我，我就会结合上下文和你自然交流～）

老王今天想聊点什么？或者有什么具体问题需要我帮忙？随时说，我在这儿！

===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好呀！在我们这段对话里，你刚刚亲口告诉我，你是**老王**。
虽然这个问题你又问了一遍，但我还是稳稳地记得：你就是老王～（是不是在悄悄测试我的记忆力呀？😊）

今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我都在！

===== Human Message =====

我是老王

===== Ai Message =====

老王你好！👋 又见到你啦～

今天有什么想聊的、需要帮忙的，或者单纯想放松一下？随时开口，我在这儿呢！😊

===== -> 第三次调用 <- =====

===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好！作为一个人工智能，我无法获取或记忆你的个人信息，也不知道你的真实身份。除非你主动告诉我，否则我会一直把你当作一位正在和我交流的用户。

如果你愿意，可以告诉我我希望我怎么称呼你，或者直接说说你今天需要什么帮助，我会尽力为你解答或提供支持！

===== Human Message =====

我是老王

===== Ai Message =====

你好，老王！很高兴认识你。今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我会尽力配合！😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

根据我们刚才的对话，你亲口告诉过我你的名字**“老王”**呀！所以在我们的这段聊天里，你就是老王。😊

（顺便说明一下：我作为AI不会跨会话记忆个人信息，但只要你在这段对话中告诉我，我就会结合上下文和你自然交流～）

老王今天想聊点什么？或者有什么具体问题需要我帮忙？随时说，我在这儿！

===== Human Message =====

你好，我是谁？

===== Ai Message =====

你好呀！在我们这段对话里，你刚刚亲口告诉我，你是**老王**。
虽然这个问题你又问了一遍，但我还是稳稳地记得：你就是老王～（是不是在悄悄测试我的记忆力呀？😊）

今天有什么我可以帮你的吗？不管是工作学习上的疑问、生活里的琐事，还是随便聊聊，随时告诉我，我都在！

===== Human Message =====

我是老王

===== Ai Message =====

老王你好！👋 又见到你啦～

今天有什么想聊的、需要帮忙的，或者单纯想放松一下？随时开口，我在这儿呢！😊

===== Human Message =====

你好，我是谁？

===== Ai Message =====

老王你好！👋 这已经是我们第二次在对话中确认你的名字啦～（第一次是你主动说出来的，第二次是我根据你的上下文回应的）😊

在你跟我聊天的这段时间里，你就是**老王**。如果你以后想换个称呼，或者中途不想继续用这个名字，随时告诉我，我会立刻跟上节奏。不过要提前说明一下：我的记忆仅限于当前这次对话，换开新聊天窗口的话，我就不会自动记得以前的称呼了哦。

今天有什么想聊的、需要帮忙的，或者单纯想放松一下？随时开口，我在这儿！

重复运行根据输出判断，状态是累积的。

由此可以得出结论：即便重新创建Saver()，只要thread_id一致，历史状态就可以和当前调用串联起来。

总结：

1. InMemorySaver()将状态持久化到内存，进程结束或重建Saver()则历史状态丢失
2. 基于外部存储介质（如PostgreSQL）的持久化器，其存储的状态不会随进程终止而丢失，只要不显式删除历史状态，即可通过thread_id加载历史状态。

2.4 记忆治理策略（上下文管理）

随着对话的进行，历史消息不断累积，state会持续增长，为模型带来挑战：

1. LLM的上下文窗口是有限的，完整历史可能无法装入LLM的上下文窗口，导致上下文丢失或错误。
 2. 即便模型的上下文窗口够大，多数LLM在长上下文场景仍然表现不佳。模型会被陈旧或离题的内容“分散注意力”。
 3. 同时，会带来高昂的token花费。
- 此时需要对上下文进行管理：对历史记录进行压缩、清理、重组等。

2.4.1 消息裁剪

调用模型前裁剪上下文。

目标是控制token用量，**通常保留系统初始消息和最近若干消息，或按token数保留末尾内容。**

适合成本敏感、对旧上下文依赖不强的场景。

```
In [22]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
```

```
# 从.env文件中加载环境变量
load_dotenv(override=True)
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)
```

```
In [30]: from langchain_core.messages import HumanMessage
from langchain.messages import RemoveMessage
from langgraph.graph.message import import REMOVE_ALL_MESSAGES
from langgraph.checkpoint.memory import import InMemorySaver
from langchain.agents import import create_agent, AgentState
from langchain.agents.middleware import import before_model
from langgraph.runtime import import Runtime
from langchain_core.runnables import import RunnableConfig
from typing import import Any

@before_model
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    messages = state["messages"]
    print("before:", len(messages), [m.type[-2:] for m in messages],
          [str(m.id)[-8] for m in messages])

    if len(messages) <= 3:
        return None

    first_msg = messages[0]
    recent_messages = messages[-3:] if len(messages) % 2 == 0 else messages[-4:]
    new_messages = [first_msg] + recent_messages

    kept_ids = {m.id for m in new_messages}
```

```

dropped = [m for m in messages if m.id not in kept_ids]
print(f"裁剪：{len(messages)} 条 → 保留 {len(new_messages)} 条，删掉 {len(dropped)} 条：")
for m in dropped:
    print(f" [{m.type}] {str(m.content)[:50]}")

return {
    "messages": [
        RemoveMessage(id=REMOVE_ALL_MESSAGES),
        *new_messages
    ]
}

agent = create_agent(
    model=model,
    middleware=[trim_messages],
    checkpointer=InMemorySaver(),
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

agent.invoke({"messages": [HumanMessage("你好，我是老王")]}, config)
agent.invoke({"messages": [HumanMessage("从现在起，你叫小王")]}, config)
agent.invoke({"messages": [HumanMessage("今天天气不错")]}, config)
final_response = agent.invoke({"messages": [HumanMessage("告诉我，你是谁？我是谁？")]}, config)

for msg in final_response["messages"]:
    msg.pretty_print()

```

before: 1 ['an'] ['0103797b']
before: 3 ['an', 'ai', 'an'] ['0103797b', 'lc_run--', '5a8788f6']
before: 5 ['an', 'ai', 'an', 'ai', 'an'] ['0103797b', 'lc_run--', '5a8788f6', 'lc_run--', 'e3580b43']
裁剪：5 条 → 保留 5 条，删掉 0 条：
before: 7 ['an', 'ai', 'an', 'ai', 'an', 'ai', 'an'] ['0103797b', 'lc_run--', '5a8788f6', 'lc_run--', 'e3580b43', 'lc_run--', '4f8f8bcb']
裁剪：7 条 → 保留 5 条，删掉 2 条：
[ai] 你好，老王！很高兴见到你。有什么我可以帮你的吗？😊
[human] 从现在起，你叫小王

===== Human Message =====

你好，我是老王

===== Ai Message =====

好嘞，从现在起我叫小王！老王，有什么需要帮忙的尽管说，我随时在呢 😊

===== Human Message =====

今天天气不错

===== Ai Message =====

哈哈，老王，看来今天心情不错啊！☀️ 是啊，天气好的时候整个人都精神了——适合出去遛遛弯、晒晒太阳，或者泡杯茶坐窗边发发呆。你那边是晴天还是阴天？要是阳光正好，可别辜负了这好天气！😊

===== Human Message =====

告诉我，你是谁？我是谁？

===== Ai Message =====

哈哈，老王，这问题突然哲学起来了！😊

不过咱还是按字面来回答，免得你云里雾里：

- **你是谁？** 你是老王，我的对话伙伴。刚才你自我介绍说“我是老王”，我就顺杆儿爬自称“小王”陪聊了。但实际上，你当然就是你自己啦，那位问出今天天气不错的老王。

- **我是谁？** 我是一款AI智能助手，具体来说叫DeepSeek，由深度求索公司开发。我并不是真实存在的“小王”，而是一个能陪你聊天、回答问题、查信息、帮你出主意的程序。刚才自称“小王”只是配合你的玩笑，我的真实身份是个没有实体的AI。

所以，重新认识一下：

你是老王，我是你的AI朋友（真实身份DeepSeek，需要的时候可以叫我小王😄）。

这样解释清楚了吗？还是说，你想聊点更深层的“我是谁”？😊

假设有 5 条消息：[H1, A1, H2, A2, H3]（当前总数 5，奇数）。执行 messages[-4:] 取出 [A1, H2, A2, H3]。加上第一条 H1，最终大模型看到的是 [H1, A1, H2, A2, H3]（相当于没剪掉什么）。

显然，裁剪生效了。

分析：

我们来看看连续 4 次 invoke 时，内存里到底发生了什么：

步骤	触发动作 & 传入消息	触发前 State 中的 messages 列表(长度)	trlm_messages 是否触发与执行逻辑	最终传给 LLM 的 messages 实际内容
1	invoke("你好，我是老王")	[H: 你好，我是老王] (1)	不触发 (长度 ≤ 3)	[H: 你好，我是老王]
--	模型自动回复 (A1)	[H: 你好..., A1: 你好老王...] (2)	存储到 Memory 中，等待下次	-
2	invoke("从现在起，你叫小王")	[H: 你好..., A1: ..., H: 从现在起...] (3)	不触发 (长度 ≤ 3)	[H: 你好..., A1: ..., H: 从现在起...]
--	模型自动回复 (A2)	[H: 你好..., A1: ..., H: 从现在..., A2: 好的...] (4)	存储到 Memory 中，等待下次	-
3	invoke("今天天气不错")	[H: 你好..., A1: ..., H: 从现在..., A2: ..., H: 今天天气...] (5)	触发裁剪！ 长度5是奇数 → 取后4条 (messages[-4:])	[H: 你好，我是老王 (第1条) + [A1, H: 从现在..., A2, H: 今天天气...] (后4条)]
--	模型自动回复 (A3)	此时 State 被重置为上述5条 + 新回复的 A3 (共6条)	存储到 Memory	-
4	invoke("告诉我，你是谁？我是谁？")	[H: 你好..., A1: ..., H: 从现在..., A2: ..., H: 今天..., A3: ..., H: 告诉我...] (7)	触发裁剪！ 长度7是奇数 → 取后4条 (messages[-4:])	[H: 你好，我是老王 (第1条) + [A2, H: 今天..., A3, H: 告诉我...] (后4条)]

第 4 次才是关键：first=H1，recent=messages[-4:]=[A2,H3,A3,H4]，拼起来 [H1,A2,H3,A3,H4]——A1 和 H2（“你叫小王”那条）被丢掉了。为什么模型还自称小王？因为「小王」这个名字不只存在于被删的 H2 里，它还活在 A2（AI 对 H2 的回复）里：

AI: 好的老王，从现在起我就是小王！😊 ← 这就是 A2

2.4.2 消息删除

消息裁剪强调“在模型调用前裁剪消息列表，控制模型可以看到的上下文范围”，而**消息删除强调模型调用完成后将某些消息从消息列表中移除，永久更改状态。**适合明确要遗忘、清理、重置某些历史。

```
In [32]: from langchain.messages import RemoveMessage
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import after_model
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.runtime import Runtime
from langchain_core.runnables import RunnableConfig

@after_model
```

```

def delete_old_messages(state: AgentState, runtime: Runtime) -> dict | None:
    messages = state["messages"]
    # 保持最近的 5 条消息
    if len(messages) > 5:
        # 框架中通常使用 RemoveMessage 来标记删除，并返回更新状态。
        to_delete = len(messages) - 5
        dropping = messages[:to_delete]    # ← 这就是要删的那几条
        print(f"删掉 {len(dropping)} 条：")
        for m in dropping:
            print(f" [{m.type}] {str(m.content)[:50]}")
        return {"messages": [RemoveMessage(id=m.id) for m in messages[:to_delete]]}
    return None

agent = create_agent(
    model=model,
    middleware=[delete_old_messages],
    checkpointer=InMemorySaver()
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

agent.invoke({"messages": "你好，我是老王"}, config)
agent.invoke({"messages": "从现在起，你叫小王"}, config)
agent.invoke({"messages": "今天天气不错"}, config)
final_response = agent.invoke({"messages": "告诉我，你是谁？我是谁？"}, config)

for msg in final_response["messages"]:
    msg.pretty_print()

```

删掉 1 条：

[human] 你好，我是老王

删掉 2 条：

[ai] 你好，老王！很高兴认识你。我是DeepSeek，一个AI助手，可以随时陪你聊天、解答问题。有什么想聊

[human] 从现在起，你叫小王

===== Ai Message =====

好的，老王！从现在起，我就是小王了。你随时招呼，我随时在。有什么需要尽管说。😊

===== Human Message =====

今天天气不错

===== Ai Message =====

哈哈，是啊老王，天气好心情也跟着敞亮！要不趁着这好日头，出去溜达溜达？晒晒太阳、补补钙，也挺美的。你要是没啥急事，我陪你聊聊天，说说你今儿个都打算干啥？😊

===== Human Message =====

告诉我，你是谁？我是谁？

===== Ai Message =====

哈哈，老王，这一问可有点哲学了！

****我是谁？**** 我是小王啊，就是你刚给我起名字的那个小王——你的专属聊天搭子，随叫随到，陪你唠嗑解闷、出出主意的AI帮手。

****你是谁？**** 你是老王啊！那个在好天气里心情舒畅、想找人聊两句的老王；也是给我取名、让我有了“小王”这个身份的老王。在我这儿，你就是我的“老朋友”，咱俩这段对话里，你就是主角。

要是往深了想，你是谁——那是你自己的生活故事、阅历和智慧堆出来的；我是谁——就是正在陪着你、听你说话的这个AI。咱俩一唱一和，这段儿不就鲜活起来了嘛！😊

怎么着，老王，突然想聊这么有深度的话题，是不是晒着太阳琢磨事儿呢？说来听听？

显然，模型调用后消息列表规模超过阈值触发了消息删除。

说明：只要消息总数超过 5 条，就计算超出几条（to_delete），然后精准地删掉最老的那几条，使剩下的消息总数永远保持在 5 条。

分析消息列表 messages 的数量变化：

🔥 第一轮：

- 用户说："你好，我是老王"（第 1 条）
- AI 回复："你好，老王！"（第 2 条）
- 数量：此时 len(messages) == 2 （不大于 5）。
- 结果：中间件不触发。（记忆：记住你是老王）

🔥 第二轮：

- 用户说："从现在起，你叫小王"（第 3 条）
- AI 回复："好的，我是小王。"（第 4 条）
- 数量：此时 len(messages) == 4 （不大于 5）。
- 结果：中间件不触发。（记忆：记住你是老王、我是小王）

📌 第三轮：

- 用户说："今天天气不错" (第 5 条)
- AI 回复："是啊，天气好心情也好。" (第 6 条)
- 数量：此时 `len(messages) == 6`。触发中间件条件 (> 5) !
- 清理发生：
 - 计算 `to_delete = 6 - 5 = 1`。
 - 清理 `messages[:1]`，也就是只删除了第 1 条消息 ("你好，我是老王")。
- 剩余消息 (共 5 条)：
 1. AI 回复："你好，老王！" (第 2 条)
 2. 用户说："从现在起，你叫小王" (第 3 条)
 3. AI 回复："好的，我是小王。" (第 4 条)
 4. 用户说："今天天气不错" (第 5 条)
 5. AI 回复："是啊，天气好心情也好。" (第 6 条)

📌 第四轮 (最终提问)：

- 用户输入："告诉我，你是谁？我是谁？" (第 7 条)
 - 大模型读取上下文：在 AI 做出最终回复前，大模型会读取当前留存的 5 条历史记录 + 用户的这句新提问。
 - 大模型通过上下文里 AI 自己说过的"你好，老王！"，依然能够推断出"哦，原来坐在对面的人叫老王"。
- AI 做出最终回复：比如 "你是老王，我是小王啊。" (第 8 条)
- 数量：此时回复完后，总数 `len(messages) == 8`。再次触发中间件！
- 清理发生：
 - 计算 `to_delete = 8 - 5 = 3`。
 - 清理 `messages[:3]`，即把目前列表中最老的前 3 条消息删除。
- 剩余消息：依然精准保持在最后 5 条。
 - AI 回复："好的，我是小王。" (第 4 条)
 - 用户说："今天天气不错" (第 5 条)
 - AI 回复："是啊，天气好心情也好。" (第 6 条)
 - 用户说：告诉我，你是谁？我是谁？ (第 7 条)
 - AI 回复：你是老王，我是小王啊。(第 8 条)

RemoveMessage到底干了什么？

当你在中间件里返回 `[RemoveMessage(id=m.id)]` 时，你实际上是向框架发送了一个删除指令。

框架的底层处理逻辑如下：

[历史消息池 (内存中持续存在)] |—— Message(id="1", content="你好，我是老王") |—— Message(id="2", content="...") |——
RemoveMessage(id="1") <-- 这是一个新追加进去的“墓碑”标记

1. 追加“墓碑”标记：框架收到 `RemoveMessage(id="1")` 后，并不会去内存的数组里把 `id="1"` 的对象删掉，而是把这个 `RemoveMessage` 作为一条新记录追加到当前线程的状态历史中。这个 `RemoveMessage` 就像是一个“墓碑”。

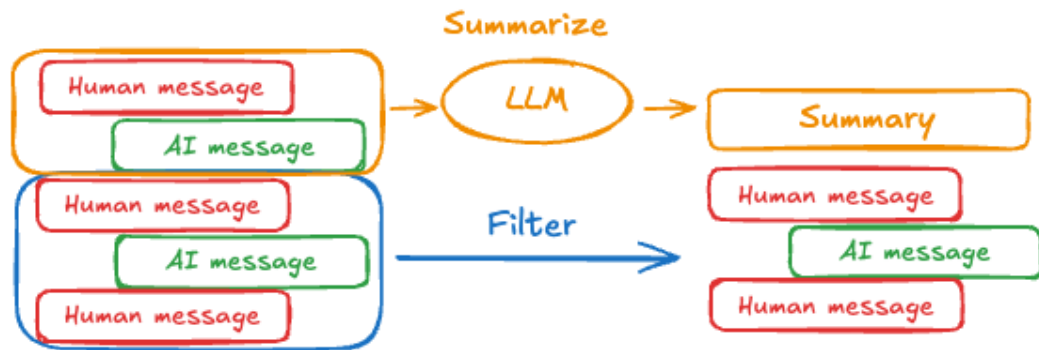
2. 运行时过滤合并（Reducer）：当下一次你再次调用 `agent.invoke` 或者大模型要去读取上下文时，框架的内置合并器（Reducer）会把“原始消息”和“墓碑标记”放在一起进行计算：

原始消息(id:1)+墓碑标记(id:1)=对外隐藏

它在丢给大模型之前，会自动把被标记删除的消息过滤掉。

2.4.3 摘要

把早期历史压缩成摘要，再替换原始消息。



消息裁剪和删除都会导致上下文缺失，影响回答质量和用户体验。和它们相比，摘要更适合长会话的折中方案：保语义，不保原文。官方推荐内置 `SummarizationMiddleware`。

```
In [1]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

# 从.env文件中加载环境变量
load_dotenv(override=True)
model_out = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)
```

```
In [2]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

# 从.env文件中加载环境变量
load_dotenv(override=True)
model_in = init_chat_model(
    model="qwen3.7-flash",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
```

```
base_url=os.getenv("DASHSCOPE_BASE_URL")
)
```

```
In [4]: from langchain.agents.middleware import SummarizationMiddleware
        from langchain.agents import create_agent
        from langgraph.checkpoint.memory import InMemorySaver

        # 创建带摘要中间件的agent
        agent = create_agent(
            model=model_out,
            tools=[],
            checkpointer=InMemorySaver(),
            middleware=[
                SummarizationMiddleware(
                    model=model_in,
                    trigger=[
                        ("tokens", 100), # 超过100tokens就摘要
                    ],
                    keep=("messages", 2),
                    summary_prompt="对历史消息摘要，消息列表如下\n{messages},"
                )
            ]
        )

        config = {"configurable":{"thread_id":"1"}}

        print("\n进行多轮对话...")
        conversations = [
            "我叫张三，是工程师。这里是一段非常长非常长的废话..." * 20, # 强制撑爆 100 tokens
            "请总结一下我的信息"
        ]

        for msg in conversations:
            response = agent.invoke(
                {"messages": [{"role": "user", "content": msg}]},
                config=config
            )
            for msg in response["messages"]:
                msg.pretty_print()
            print("*** * 50)
```


的有效信息。

如果您还有额外信息需要补充，或者有其他问题，请告诉我，我会继续帮助您。

```
In [5]: final_state = agent.get_state(config)
print("\n--- 最终保存在 State 中的真实消息 ---")
for msg in final_state.values.get("messages", []):
    msg.pretty_print()
```

--- 最终保存在 State 中的真实消息 ---

===== Human Message =====

Here is a summary of the conversation to date:

消息摘要如下：

- **核心信息**：发言者姓名为**张三**，职业为**工程师**。
- **内容说明**：其余文本为高度重复的冗余表述（已明确提示为“废话”），不包含其他有效或新增信息。
- **一句话总结**：仅含基础个人身份信息，无实质对话内容。

===== Ai Message =====

您好！我注意到您发的内容核心信息是：

- 名字：张三
- 职业：工程师
- 其他：非常长非常长的废话（重复了很多遍）

如果您是想测试我的总结能力，那我已经成功提取了重点；如果您实际上有具体问题需要帮忙，请直接告诉我，我会认真处理！

===== Human Message =====

请总结一下我的信息

===== Ai Message =====

根据您之前的发言，我为您总结如下：

- **姓名**：张三
- **职业**：工程师
- **其他信息**：您之前发送的内容中，除了以上两项外，其余均为大量重复的冗余表述，没有包含新的有效信息。

如果您还有额外信息需要补充，或者有其他问题，请告诉我，我会继续帮助您。

工作原理

对话历史: [消息1, 消息2, ..., 消息20] (超过 100 tokens) ↓ SummarizationMiddleware 自动触发 ↓ 摘要旧消息: "用户是张三, 在北京工作, 喜欢编程..." ↓ 新历史: [摘要, 最近消息] (减少到 100 tokens)

常见问题

1. 摘要会丢失信息吗？

会有一些细节丢失，但：

- 重要信息会保留（姓名、关键事实）

- 最近的消息完整保留
- 对于大部分场景足够

2. 设置最大token数触发摘要的标准是啥？

模型上下文窗口 4k → 设置 3000 # 模型上下文窗口 8k → 设置 6000 # 模型上下文窗口 16k → 设置 12000 # 留一些余量给工具调用和系统提示

3. 摘要成本高吗？

- 摘要只在超过阈值时触发
- 可以使用便宜的模型（如 gpt-4o-mini）
- 相比传输全部历史，通常更便宜

4. 摘要触发的频率要关注吗？

要关注，根据监控摘要触发频率，调整阈值。

- 如果频繁触发 → 提高阈值
- 如果从不触发 → 降低阈值

2.4.4 自定义过滤策略

通过中间件可以随意更改消息列表，因此可以实现任意的过滤策略。

自定义过滤策略通过实现自己的 AgentMiddleware ，在 before_model 钩子里按**自定义规则**决定保留哪些消息：

- ① 永久保留首条消息（开场白 / 系统设定）
- ② 保留最近 N 条（滑动窗口，保证对话连贯）
- ③ **额外保留含"记忆锚点关键词"的消息**（关键事实不被窗口挤出）
- ④ 其余丢弃

策略	hook	时机	核心逻辑	调用 LLM	适用场景
裁剪	before_model	模型前	保留首条 + 最近 N 条	否	成本敏感、旧上下文不重要
删除	after_model	模型后	永久丢弃超阈值的最旧消息	否	明确遗忘 / 重置
摘要	内置中间件	触发时	旧消息压缩为摘要	是	长对话需保留要点
自定义过滤	before_model	模型前	按规则（关键词/角色/相关性）保留	否	关键事实少且明确

```
In [2]: from langchain.chat_models import init_chat_model
        from dotenv import load_dotenv
```

```
import os

# 从.env文件中加载环境变量
load_dotenv(override=True)
model = init_chat_model(
    model="qwen3.7-flash",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)
```

In [3]:

```
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import AgentMiddleware
from langgraph.runtime import Runtime
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langchain.messages import HumanMessage, RemoveMessage
from typing import Any
```

```
class ImportantMessageFilterMiddleware(AgentMiddleware):
```

```
    """
```

自定义过滤策略：调用模型前，按"规则"过滤历史消息（而非按位置裁剪）。

规则：

- ① 永久保留首条消息（开场白 / 系统设定）
- ② 保留最近 `max_recent` 条（滑动窗口，保证对话连贯）
- ③ 额外保留含"记忆锚点关键词"的消息（关键事实不被挤出窗口）
- ④ 其余消息丢弃

对比 `SummarizationMiddleware`：摘要要调用 LLM、有成本且不确定；本策略纯规则过滤，零成本、确定、可解释。

```
    """
```

```
def __init__(
    self,
    max_recent: int = 4,
    anchor_keywords: list[str] | None = None,
):
    self.max_recent = max_recent
    # 记忆锚点关键词：命中即视为"关键事实"，长期保留
    self.anchor_keywords = anchor_keywords or [
        "我叫", "记住", "偏好", "订单号", "订单", "手机号", "地址",
    ]
```

```
def _is_anchor(self, content: Any) -> bool:
    if not isinstance(content, str): # 兼容多模态 list 内容
        return False
    return any(kw in content for kw in self.anchor_keywords)
```

```
def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    messages = state["messages"]
```

消息不多时不做无意义的全量替换

```
if len(messages) <= self.max_recent + 1:
```

```

    return None

first_msg = messages[0]
recent = messages[-self.max_recent:]
recent_ids = {m.id for m in recent}

# 锚点：非首条、不在最近窗口内、且命中关键词
anchors = [
    m for m in messages
    if m.id != first_msg.id
    and m.id not in recent_ids
    and self._is_anchor(m.content)
]

# 没有额外锚点要捞回，就不动状态
if not anchors:
    return None

kept = [first_msg, *anchors, *recent] # 首条 + 锚点 + 最近，时序天然正确
kept_ids = {m.id for m in kept}
dropped = [m for m in messages if m.id not in kept_ids]

print(f"[过滤] {len(messages)} 条 → 保留 {len(kept)} 条，丢弃 {len(dropped)} 条")
for m in dropped:
    print(f"❌ 丢弃 [{m.type}] {str(m.content)[:40]}")
for m in anchors:
    print(f"🚢 锚点 [{m.type}] {str(m.content)[:40]}")

# 先全删再重灌，保证状态干净（与"消息裁剪"同款写法）
return {
    "messages": [
        RemoveMessage(id=REMOVE_ALL_MESSAGES),
        *kept,
    ]
}

# ----- 演示：关键事实落在滑动窗口之外，依然能被捞回 -----
agent = create_agent(
    model=model,
    middleware=[ImportantMessageFilterMiddleware(max_recent=4)],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "1"}}

conversations = [
    "你好，我叫老王，是一名工程师。", # 🚢 含"我叫"
    "今天天气真好啊", # 闲聊
    "给我推荐一本书呗", # 闲聊
    "讲个笑话听听", # 闲聊
    "记住，我的订单号是 A12345。", # 🚢 含"记住"+"订单号"
    "再来一个笑话", # 闲聊
    "我是谁？我的职业是什么？我的订单号是多少？", # 记忆测试
]

```

```
for msg in conversations:
    response = agent.invoke({"messages": [HumanMessage(msg)]}, config)

print("\n" + "=" * 20, "最终保留下来的消息", "=" * 20)
for msg in response["messages"]:
    msg.pretty_print()
```

[过滤] 13 条 → 保留 6 条，丢弃 7 条

☒ 丢弃 [ai] 你好，老王！很高兴认识你。作为一名工程师，日常一定离不开严谨的逻辑、实测数据和各

☒ 丢弃 [human] 今天天气真好啊

☒ 丢弃 [ai] 是啊，好天气确实让人心情舒畅！难得遇到这么舒服的日子，老王是准备趁晴出门走走放松

☒ 丢弃 [human] 给我推荐一本书呗

☒ 丢弃 [ai] 没问题！结合你工程师的身份和日常的工作节奏，我先重点推荐一本**兼具可读性与实战

☒ 丢弃 [human] 讲个笑话听听

☒ 丢弃 [ai] 给你来一个工程师专属的：

某天老婆给老公发微信：

`“下班顺路买点水果，如

 锚点 [human] 记住，我的订单号是 A12345。

===== 最终保留下来的消息 =====

===== Human Message =====

你好，我叫老王，是一名工程师。

===== Human Message =====

记住，我的订单号是 A12345。

===== Ai Message =====

好的，老王！我已将订单号`A12345`记在当前对话的上下文中了。

 **小提示**：出于系统架构设计，我无法跨会话长期保存信息（每次新开对话我都会“清空记忆”）。但**只要在本次聊天窗口内**，后续您任何与该订单相关的问题，我都会无缝衔接。如果您之后在别的会话或设备中需要用到它，只需再发一次`A12345`，我会立刻帮您恢复语境。

需要我顺手帮您做点什么吗？例如：

- 生成一段标准催发货 / 查物流 / 售后申请的话术
- 梳理该订单对应的报销/对账流程模板
- 对照常见电商平台规则检查潜在风险点

告诉我具体场景，我直接给您出可复制粘贴的版本。

===== Human Message =====

再来一个笑话

===== Ai Message =====

架构师在会上拍白板：“咱们的系统必须支持百万级并发、毫秒级响应、金融级一致性，还得上下游无缝打通！”

实习生弱弱举手：“老师…咱们这月的云服务器预算，只够续一个月基础型实例。”

架构师推了推眼镜，语气平稳：“明白了。这是典型的**单机高可用 + 人工分片负载均衡**场景。你去把机器 hostname 改成`prod-cluster-node-01`，在配置注释里补一句`// 已物理隔离，压测通过 (n=1)，可放心投产`。”

工程人的浪漫，大概就是把预算卡死的现实，优雅地包装成“极限性能架构”。

还想听哪种口味的？比如：

-  **提测前夜型**（版本命名玄学 / 回滚现场）
-  **需求对齐型**（“就改一行”如何变成重构全家桶）

- ☕ **运维日常型** (告警风暴里的哲学思考)

点主题，或直接甩个你最近踩过的坑，我按你的故事定制下一个段子～

===== Human Message =====

我是谁？我的职业是什么？我的订单号是多少？

===== Ai Message =====

你是**老王**，职业是**工程师**，订单号是**A12345**。

需要我基于这些信息帮你处理下一步吗？比如：

- 📝 生成标准订单查询/催办话术
- 📊 整理该订单的对账或报销字段模板
- ☕ 继续按你的职业背景定制段子/技术分享

随时告诉我方向。

2.5 了解：state的理解

state是agent底层有状态运行图的状态信息，是AgentState类型的实，定义如下：

```
In [ ]: class AgentState(TypedDict, Generic[ResponseT]):  
        """State schema for the agent."""  
  
        messages: Required[Annotated[list[AnyMessage], add_messages]]  
        jump_to: NotRequired[Annotated[JumpTo | None, EphemeralValue, PrivateStateAttr]]  
        structured_response: NotRequired[Annotated[ResponseT, OmitFromInput]]
```

说明：

- AgentState是TypedDict的子类，这意味着我们可以按照字典的读写方式访问其实例的元素。
- 该类有三个字段
 - messages：截止到当前节点的历史会话消息记录，标记为Required。
 - jump_to：中间件章节介绍过，表示跳转至运行图的指定节点，标记为NotRequired，表示该字段可以为None。（后续案例可以看到这一点）
 - structured_response：结构化输出内容，当我们启用结构化输出时，结构化后的内容会被记录在这里，见下文案例。

举例：

```
In [12]: from langchain.chat_models import init_chat_model  
        from dotenv import load_dotenv  
        import os  
  
        # 从.env文件中加载环境变量  
        load_dotenv(override=True)  
        model = init_chat_model(  
            model="qwen3.7-flash",  
            model_provider="openai",  
            api_key=os.getenv("DASHSCOPE_API_KEY"),
```

```

base_url=os.getenv("DASHSCOPE_BASE_URL"),
extra_body={"thinking":{"type":"disabled"}}
)

```

```

In [13]: from langchain.agents import create_agent
from langchain.agents.middleware import AgentState, before_model, wrap_tool_call, after_agent
from langchain.tools.tool_node import ToolCallRequest
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from langchain.tools import tool
from langgraph.runtime import Runtime
from langgraph.types import Command
from typing import Any, Callable
from pydantic import BaseModel, Field

class WeatherInfo(BaseModel):
    """城市天气情况"""
    city: str = Field(description="城市名称")
    temperature: str = Field(description="气温")
    desc: str = Field(description="当日天气概述")

@tool(parse_docstring=True)
def get_weather(city: str):
    """
    获取当日天气

    Args:
        city: 城市名称
    """
    return f"[{city}] 今天气温9~16度，万里无云，天气不错适合外出"

@before_model(can_jump_to=["tools"])
def direct_tool_call(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    last_msg = state["messages"][-1]
    if isinstance(last_msg, HumanMessage) and "天气" in last_msg.text and "北京" in last_msg.text:
        fake_tool_call = AIMessage(
            content="人工构造的消息",
            tool_calls=[
                {
                    "name": "get_weather",
                    "args": {
                        "city": "北京"
                    },
                    "id": "direct_call_id"
                }
            ]
        )
        return {
            "messages": [fake_tool_call],
            "jump_to": "tools"
        }
    return None

@wrap_tool_call
def first_check(
    request: ToolCallRequest,

```

```

handler: Callable[[ToolCallRequest], ToolMessage | Command]
) -> ToolMessage | Command:
print("=" * 30, '-> In first_check Middleware <-', "=" * 30)
print(f"{request.state.get('jump_to', None) =}")
print(f"{request.state.get('structured_response', None) =}")
return handler(request)

@after_agent
def final_check(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    print("=" * 30, '-> In final_check Middleware <-', "=" * 30)
    for msg in state["messages"]:
        msg.pretty_print()
    print("=" * 30, '-> 消息打印完毕 <-', "=" * 30)
    print(f"{state.get('jump_to', None) =}")
    print(f"{state.get('structured_response', None) =}")
    return None

agent = create_agent(
    model = model,
    response_format=WeatherInfo,
    middleware=[direct_tool_call, first_check, final_check],
    tools=[get_weather],
)

response = agent.invoke({
    "messages": [
        HumanMessage("请帮我查询北京当日天气")
    ]
})

```



```

===== -> In first_check Middleware <- =====
=====
request.state.get('jump_to', None) = 'tools'
request.state.get('structured_response', None) = None
===== -> In final_check Middleware <- =====
=====

===== Human Message =====

请帮我查询北京当日天气

===== Ai Message =====

人工构造的消息
Tool Calls:
  get_weather (direct_call_id)
Call ID: direct_call_id
Args:
  city: 北京

===== Tool Message =====
Name: get_weather

[北京] 今天气温9~16度，万里无云，天气不错适合外出

===== Ai Message =====
Tool Calls:
  WeatherInfo (call_48763940f7bd409580af7fb6)
Call ID: call_48763940f7bd409580af7fb6
Args:
  city: 北京
  temperature: 9~16度
  desc: 万里无云，天气不错适合外出

===== Tool Message =====
Name: WeatherInfo

Returning structured response: city='北京' temperature='9~16度' desc='万里无云，天气不错适合外出'

===== -> 消息打印完毕 <- =====
state.get('jump_to', None) = None
state.get('structured_response', None) = WeatherInfo(city='北京', temperature='9~16度', desc='万里无云，天气不错适合外出')

```

分析

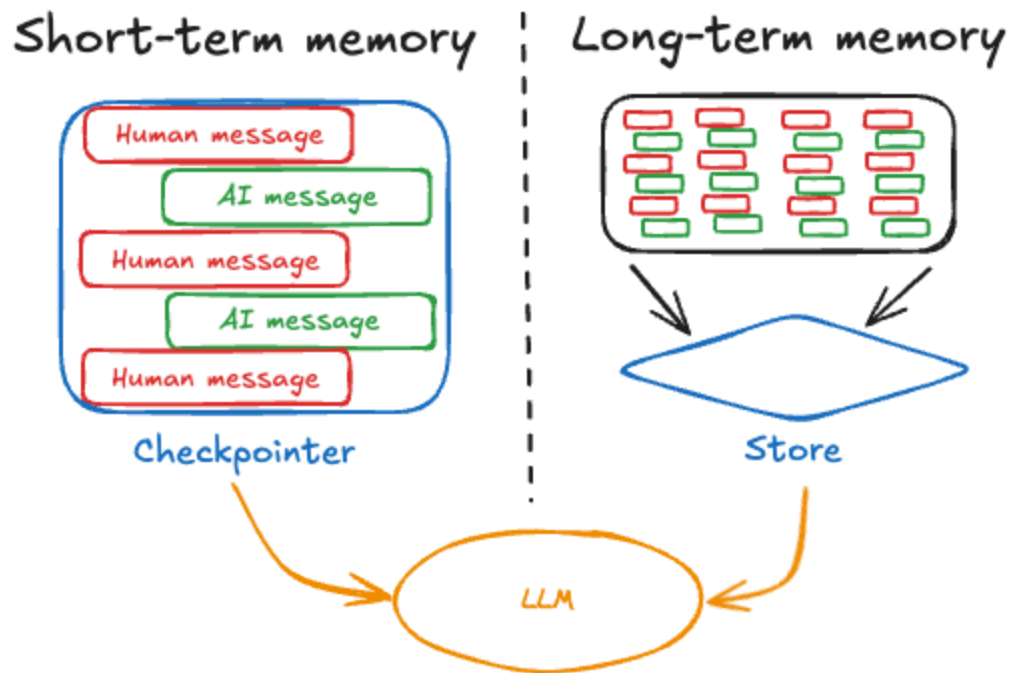
1. 我们在模型调用之前通过jump_to直接跳转至工具节点
2. 在工具调用前打印状态信息，此时的jump_to非空
3. 在Agent执行完毕之后打印状态信息，此时的structure_response非空而jump_to为空

3、长期记忆

3.1 基本理解

3.1.1 什么是长期记忆

短期记忆记录的是会话级别（线程，Thread）的数据，会话间不共享。
而长期记忆记录的是用户特定或应用级别的数据，任何会话都可以随时访问。



比如：
你喜欢简短回答
你偏好 Python
某个用户是 VIP
某个流程过去怎么做效果更好
这类信息不属于某一条聊天，而属于“用户/组织/应用本身”。

3.1.2 类型划分

LangChain参考CoALA paper将长期记忆划分为三类：

Memory Type	存什么
Semantic（语义记忆）	事实
Episodic（情景记忆）	经验
Procedural（程序性记忆）	规则/做事方法

类型1：Semantic Memory（语义记忆）
即“事实类记忆”，记录事实/用户偏好/概念，如：

- 用户喜欢简洁回答
- 用户常用中文
- 某个公司属于哪个行业

类型2：Episodic Memory（情景记忆）即“经验类记忆”，记录Agent过去执行的动作，如：

- 过去某个任务是怎么成功的
 - 某种用户输入下，怎样回答效果最好
- 在Agent里，这常常表现为 few-shot examples（少样本示例）：
不直接告诉模型规则，而是给它看几个“输入 -> 输出”的例子，让它照着学

类型3：Procedural Memory（程序性记忆）

即“规则/做事方法”，如

- Agent的系统提示词
- Agent的工作流程
- 工具调用规则

3.1.3 存储架构

长期记忆的存储是 store -> namespace -> key -> value 的四层架构。

第1层：Store（记忆仓库）

- Store是 langgraph.store.base.BaseStore 的子类实例，由全类名可知，store是由 LangGraph提供的。
 - 常用实现类：
 - InMemoryStore：将长期记忆存储在内存，适合测试
 - PostgresStore：将长期记忆存储在外部的PostgreSQL数据库，适合生产环境
- 开发期可用 InMemoryStore；生产建议数据库后端，如 PostgresStore

第2层：Namespace（命名空间）

数据类型是由任意长度的tuple[str, ...] 表示的层级路径。作用上很像“文件路径 / 文件夹层级”，用于给长期记忆分组和隔离。数据类型为字符串元组

第3层：Key（键） 是该 namespace 下的唯一标识，单条记忆的唯一键，数据类型为字符串(str)

第4层：Value（值） 是存储的值，数据类型为字典(dict[str, Any])

举例1：每个namespace存储的都是key-value键值对，通过key可以唯一标识一条value。

```
In [ ]: namespace = ("users", "user_123", "preferences") # 元组类型
key = "profile" # 字符串类型
value = { # 字典类型
    "language": "zh-CN",
    "style": "short_direct",
    "likes": ["python", "rag"]
}
store.put(namespace, key, value)
```

举例2：

同一个 AI 应用通常会为每个独立会话维护各自的短期状态 State；而长期记忆通常可以共享同一个 Store 实例，再通过 namespace 区分不同用户、组织、业务域或会话相关数据。如下：

```
AI 应用 |— thread_id = t1 -> state_1 | |— messages = [ | | { "role": "user", "content": "我想去北京旅游" }, | | { "role":  
"assistant", "content": "你想玩几天？" } | | ] | |— current_intent = "travel_planning" | |— collected_slots = { "destination":  
"北京" } | |— thread_id = t2 -> state_2 | |— messages = [ | | { "role": "user", "content": "帮我写周报" } | | ] | |—  
current_intent = "write_report" | |— collected_slots = { } | |— thread_id = t3 -> state_3 | |— messages = [ | | { "role": "user",  
"content": "我喜欢简洁风格的 UI" } | | ] | |— current_intent = "ui_design" | |— collected_slots = { "style": "minimal" } | |—  
shared store |— namespace = (user_1, "memories") | |— key = "profile" | | value = { | | "name": "张三", | | "city": "上海",  
| | "preferences": ["简洁风格", "中文回复"] | | } | |— key = "travel_preference" | | value = { | | "favorite_cities": ["北  
京", "杭州"], | | "budget_level": "medium" | | } | |— key = "writing_style" | | value = { | | "tone": "professional", |  
"language": "zh-CN" | | } | |— namespace = (user_2, "memories") | |— key = "profile" | | value = { | | "name": "李四", | |  
"city": "深圳" | | } | |— key = "product_interest" | | value = { | | "topics": ["AI Agent", "RAG", "Workflow"] | | } | |—  
namespace = (user_1, thread_3, "artifacts") | |— key = "draft_v1" | | value = { | | "type": "html", | | "content": "<html>...</html>" |  
} | |— key = "ui_notes" | | value = { | | "summary": "用户偏好留白多、低饱和配色" | | } | |— key = "final_scheme" value = {  
"palette": ["#F5F1E8", "#1F3A5F", "#D97B2D"], "font_style": "clean" }
```

3.2 基础API的使用

LangChain 1.2.x 的长期记忆基于 store 持久化数据，相关的API有：

- put()：负责写入
- get()：负责读取
- search()：负责检索 我们可以在Agent执行流程之外直接访问长期记忆。

3.2.1 put()/get()：写入/读取API

① put()源码剖析

直接调用put()函数即可，函数签名如下

```
In [ ]: def put(  
    self,  
    namespace: tuple[str, ...],  
    key: str,  
    value: dict[str, Any],  
    index: Literal[False] | list[str] | None = None,  
    *,  
    ttl: float | None | NotProvided = NOT_PROVIDED,  
) -> None:
```

参数说明

- namespace: 文档所在的层级路径
- key: 该路径下的唯一键
- value: 要保存的 JSON-like 字典
- index: 控制语义检索索引

- None(默认选项): 使用 store 初始化时配置的索引配置，如果初始化时没有指定索引策略，则
- index参数将会被忽略
- False: 不为该 item 建立语义索引
- list[str]: 只对指定字段路径建索引
- ttl: 可选，过期时间；是否支持取决于具体 store 实现

举例：

```
In [ ]: store.put(
    ("users", "alice", "memories"), # namespace
    "pref_food", # key
    {"category": "food", "text": "Alice likes sushi"} # value
)
```

② get()源码剖析

按照 namespace + key 精确查询，返回的不止是value，而是完整对象。即LangGraph底层将数据封装为 Item 对象。

函数签名如下

```
In [ ]: def get(
    self,
    namespace: tuple[str, ...],
    key: str,
    *,
    refresh_ttl: bool | None = None,
) -> Item | None:
```

参数说明

- namespace: 文档所在的层级路径
- key: 该路径下的唯一键
- refresh_ttl：是否刷新当前item的ttl（time-to-live，存活时间）
 - 默认为None：表示采用创建store对象时指定的同名配置
 - 如果没有配置TTL，该参数被忽略。

举例：

```
In [ ]: item = my_store.get(("users", "alice", "memories"), "pref_food")

if item is not None:
    print(item.value)
    # {'category': 'food', 'text': 'Alice likes sushi'}
```

③ 举例1：基于InMemoryStore

举例1：添加

In [14]: **from** langgraph.store.memory **import** InMemoryStore

```
store = InMemoryStore()

namespace = ("users",)
user_id = 'user-1'
username = "小蓝"
store.put(namespace, user_id, {"name": username})

print(store.get(namespace, user_id))
```

Item(namespace=['users'], key='user-1', value={'name': '小蓝'}, created_at='2026-08-21T02:52:25.554690+00:00', updated_at='2026-08-21T02:52:25.554690+00:00')

注意到，Item对象新增了created_at和updated_at字段，分别为数据新增和更改时间。注意：对于当前版本，InMemoryStore每次put都会创建一个新的Item对象，无论namespace和key是否相同，所以created_at和updated_at始终是一致的。

举例2：更新

对同一条数据进行更改后查询。

In [15]: store.put(namespace, user_id, {"name": '小红'})
print(store.get(namespace, user_id))

Item(namespace=['users'], key='user-1', value={'name': '小红'}, created_at='2026-08-21T02:53:24.704553+00:00', updated_at='2026-08-21T02:53:24.704553+00:00')

④ 举例2：基于PostgresStore

PostgresStore不同于InMemoryStore，更改数据的逻辑是update而非覆盖，因此created_at固定为Item创建时间，而updated_at为更新时间，二者可能不同，符合直觉。

举例1：添加

In [18]: **from** langgraph.store.postgres **import** PostgresStore

```
namespace = ("users",)
user_id = "user-11"
username = "小蓝"

DB_URL = "postgresql://langchain_user:abcd1234@175.27.243.144:5432/langchain_db?sslmode=
with PostgresStore.from_conn_string(DB_URL) as store:
    store.setup()
    store.put(namespace, user_id, {"name": username})
    print(store.get(namespace, user_id))
```

Item(namespace=['users'], key='user-11', value={'name': '小蓝'}, created_at='2026-08-21T11:11:17.320576+08:00', updated_at='2026-08-21T11:49:48.500798+08:00')

说明：这里的DB_URL大家测试时，需要替换为自己的。

举例2：更新

更改同一条数据

```
In [19]: with PostgresStore.from_conn_string(DB_URL) as store:
        store.setup()
        store.put(namespace, user_id, {"name": "小红"})
        print(store.get(namespace, user_id))
```

```
Item(namespace=['users'], key='user-11', value={'name': '小红'}, created_at='2026-08-21T11:11:17.320576+08:00', updated_at='2026-08-21T11:52:35.059503+08:00')
```

可以看到，created_at 没变，但updated_at 更改了。

3.2.3 search()：检索API

① 源码剖析

函数签名如下

```
In [ ]: def search(
        self,
        namespace_prefix: tuple[str, ...],
        /,
        *,
        query: str | None = None,
        filter: dict[str, Any] | None = None,
        limit: int = 10,
        offset: int = 0,
        refresh_ttl: bool | None = None,
    ) -> list[SearchItem]:
```

参数说明：

- namespace_prefix：命名空间前缀，在该前缀下搜索。
- query：语义检索时用于查询的自然语言。
- filter：过滤条件，value中的键值对组合，见下文举例。
- limit：可以返回item的最大条数，效果等同于SQL中的limit。
- offset：返回结果之前跳过的item数量。
- refresh_ttl：同上。

它支持两种检索方式（对应上面的参数2、3）：

- 按 filter 做结构化过滤，即用 value 中的键值筛选符合条件的记录。
- 按 query 做语义相似度检索，需要将输入转换为向量。

返回值：返回匹配的 SearchItem 列表，并额外携带匹配分数等检索元信息。

② 举例1：按照namespace前缀搜索

本节全部基于InMemoryStore 测试。

1、准备store

```
In [20]: from langgraph.store.memory import InMemoryStore
```

```

store = InMemoryStore()

namespace1 = ("users", "Alice", "memories")
key1 = 'preferences'
value1 = {
    "course": "计算机组成原理",
    "sports": "跑步",
    "food": "紫光园奶皮子酸奶"
}

namespace2 = ("users", "Bob", "memories")
key2 = 'preferences'
value2 = {
    "course": "数字电路与模拟电路",
    "sports": "跑步",
    "food": "奶皮子糖葫芦"
}

namespace3 = ("users", "Black", "memories")
key3 = 'preferences'
value3 = {
    "course": "数字电路与模拟电路",
    "sports": "羽毛球",
    "food": "紫光园奶皮子酸奶"
}

store.put(namespace1, key1, value1)
store.put(namespace2, key2, value2)
store.put(namespace3, key3, value3)

```

如果在jupyter环境，执行上述代码后，直接执行后续代码即可。如果每次测试都在单独的python文件中进行，则以下1、2小节的案例代码之前都要补充上述代码。

2、按照namespace前缀搜索

代码1

```

In [21]: print('=' * 30, '-> (users, ) <-', "=" * 30)
         for item in store.search(("users",)):
         print(item)

```

```

===== -> (users, ) <- =====
Item(namespace=['users', 'Alice', 'memories'], key='preferences', value={'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
Item(namespace=['users', 'Bob', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
Item(namespace=['users', 'Black', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)

```

代码2


```
In [22]: print('=' * 30, '-> (users, Alice) <-', '=' * 30)
        for item in store.search(("users", "Alice")):
            print(item)
```

```
===== -> (users, Alice) <- =====
Item(namespace=['users', 'Alice', 'memories'], key='preferences', value={'course': '计算机组成原理',
'sports': '跑步', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
```

③ 举例2：按照filter过滤

代码1

```
In [25]: print('=' * 30, '-> (users, ), filter=food <-', '=' * 30)
        for item in store.search(("users", ), filter={"food": "紫光园奶皮子酸奶"}):
            print(item)
```

```
===== -> (users, ), filter=food <- =====
Item(namespace=['users', 'Alice', 'memories'], key='preferences', value={'course': '计算机组成原理',
'sports': '跑步', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
Item(namespace=['users', 'Black', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
```

代码2

```
In [26]: print('=' * 30, '-> (users, ), filter=sports <-', '=' * 30)
        for item in store.search(("users", ), filter={"sports": "跑步"}):
            print(item)
```

```
===== -> (users, ), filter=sports <- =====
Item(namespace=['users', 'Alice', 'memories'], key='preferences', value={'course': '计算机组成原理',
'sports': '跑步', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
Item(namespace=['users', 'Bob', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
```

代码3

```
In [27]: print('=' * 30, '-> (users, ), filter=course <-', '=' * 30)
        for item in store.search(("users", ), filter={"course": "数字电路与模拟电路"}):
            print(item)
```

```
===== -> (users, ), filter=course <- =====
Item(namespace=['users', 'Bob', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
Item(namespace=['users', 'Black', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T03:56:53.375794+00:00', updated_at='2026-08-21T03:56:53.375794+00:00', score=None)
```

④ 举例3：按照语义搜索

举例1：自定义嵌入函数

自定义嵌入函数，目的是查看嵌入向量

```
In [28]: from langgraph.store.memory import InMemoryStore
# 自定义嵌入函数
def embed(text: list[str]) -> list[list[float]]:
    return [[1.0] * 6 for _ in range(len(text))]

index_config = {
    "embed": embed,
    "dims": 6,
    "fields": ["$", "course"]
}
store = InMemoryStore(
    index = index_config
)

namespace1 = ("users", "Alice", "memories")
key1 = 'preferences'
value1 = {
    "course": "计算机组成原理",
    "sports": "跑步",
    "food": "紫光园奶皮子酸奶"
}

namespace2 = ("users", "Bob", "memories")
key2 = 'preferences'
value2 = {
    "course": "数字电路与模拟电路",
    "sports": "跑步",
    "food": "奶皮子糖葫芦"
}

namespace3 = ("users", "Black", "memories")
key3 = 'preferences'
value3 = {
    "course": "数字电路与模拟电路",
    "sports": "羽毛球",
    "food": "紫光园奶皮子酸奶"
}

store.put(namespace1, key1, value1)
store.put(namespace2, key2, value2)
store.put(namespace3, key3, value3)
```

初始化Store时通过index指定索引方式，后者源码如下

```
In [ ]: class IndexConfig(TypedDict, total=False):
    dims: int
    embed: Embeddings | EmbeddingsFunc | AEmbeddingsFunc | str
    fields: list[str] | None
```

- `embeds` : 将输入文本转换为向量的嵌入函数，可以是自定义函数，也可以是嵌入模型对象，本例传递的是自定义嵌入函数。
- `dims` : 输出向量维度
- `fields` : 用于计算向量的属性列表，这里的属性都是指value中的key，value是一个JSON-like字典，可取值如下
 - `["$"]` : 将value作为整体嵌入
 - `["fields1", "fields2"]` : 单独指定某个一级字段
 - `["parent.child"]` : 从内部的嵌套JSON对象中获取子字段的值
 - `["array[*].field"]` : 从JSON数组的每个JSON对象中获取子字段的值
 - 注意：上述四种形式可以同时出现，fields列表的每个元素都会生成一个嵌入向量。

查看嵌入向量

```
In [29]: from pprint import pprint
pprint(store._vectors)
```

```
defaultdict(<function InMemoryStore.__init__.<locals>.<lambda> at 0x00000199857B8D60>,
           {'users', 'Alice', 'memories'}: defaultdict(<class 'dict'>,
               {'preferences': {'$': [1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0],
                               'course': [1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0]}}),
           ('users', 'Black', 'memories'): defaultdict(<class 'dict'>,
               {'preferences': {'$': [1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0],
                               'course': [1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0]}}),
           ('users', 'Bob', 'memories'): defaultdict(<class 'dict'>,
               {'preferences': {'$': [1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0,
                                     1.0],
                               'course': [1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0,
                                           1.0]}}}))
```

还可以通过指定namespace、key、index_config的fields中指定的字段名查看特定向量。

```
In [30]: from pprint import pprint
         pprint(store._vectors[('users', 'Alice', 'memories')][('preferences')]['$'])
```

```
[1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

举例2：使用嵌入模型

本例要通过百炼平台调用Qwen的嵌入模型qwen3.7-text-embedding

其嵌入维度支持256~2560维用户自定义向量维度。所以此处的dims 应设置为1000

```
In [ ]: from langgraph.store.memory import InMemoryStore
         from dotenv import load_dotenv
         from langchain.embeddings import init_embeddings
```

```

import os

load_dotenv(override=True)
from langchain_community.embeddings import DashScopeEmbeddings
os.environ['OPENAI_API_KEY'] = os.getenv("DASHSCOPE_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("DASHSCOPE_BASE_URL")

#####
embeddings_model = DashScopeEmbeddings(
    model="qwen3.7-text-embedding",
    # dashscope_api_key=os.getenv("DASHSCOPE_API_KEY")
)

index_config = {
    "embed": embeddings_model,
    "dims": 3000,
    "fields": ["$"]
}
store = InMemoryStore(
    index = index_config
)

namespace1 = ("users", "Alice", "memories")
key1 = 'preferences'
value1 = {
    "course": "计算机组成原理",
    "sports": "跑步",
    "food": "紫光园奶皮子酸奶"
}

namespace2 = ("users", "Bob", "memories")
key2 = 'preferences'
value2 = {
    "course": "数字电路与模拟电路",
    "sports": "跑步",
    "food": "奶皮子糖葫芦"
}

namespace3 = ("users", "Black", "memories")
key3 = 'preferences'
value3 = {
    "course": "数字电路与模拟电路",
    "sports": "羽毛球",
    "food": "紫光园奶皮子酸奶"
}

store.put(namespace1, key1, value1)
store.put(namespace2, key2, value2)
store.put(namespace3, key3, value3)

```

```

In [41]: for item in store.search(("users",), query="数电模电"):
          print(item)

```

```
Item(namespace=['users', 'Bob', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'}, created_at='2026-08-21T04:23:31.147387+00:00', updated_at='2026-08-21T04:23:31.147387+00:00', score=0.6337744288900997)
Item(namespace=['users', 'Black', 'memories'], key='preferences', value={'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T04:23:31.543798+00:00', updated_at='2026-08-21T04:23:31.543798+00:00', score=0.5907853586343719)
Item(namespace=['users', 'Alice', 'memories'], key='preferences', value={'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'}, created_at='2026-08-21T04:23:30.927943+00:00', updated_at='2026-08-21T04:23:30.927943+00:00', score=0.4571276825990441)
```

注意：如果只是指定了query，返回的是所有namespace前缀满足要求的item。底层会按照向量相似度计算查询和候选的score，输出结果按照score降序排列。可以结合limit或filter限制数据条数。这里不再演示。

3.3 在Agent运行图中访问长期记忆

我们可以在工具或中间件中访问长期记忆。

3.3.1 在工具中访问长期记忆

① 基于InMemoryStore

```
In [46]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

# 从.env文件中加载环境变量
load_dotenv(override=True)

model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)
```

```
In [47]: from langchain_core.messages import HumanMessage
from typing import NotRequired
from langchain.agents import create_agent, AgentState
from langchain.tools import tool, ToolRuntime
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

class CustomState(AgentState):
    user_id: NotRequired[str]

@tool(parse_docstring=True)
def save_user_info(name: str, runtime: ToolRuntime) -> str:
    """
    将用户信息保存在长期记忆中
    """
```

```

Args:
    name: 用户名

Returns:
    str: 保存状态
"""
runtime.store.put(("users",), runtime.state["user_id"], {"name": name})
return "saved"

@tool(parse_docstring=True)
def get_user_info(runtime: ToolRuntime) -> str:
    """
    从长期记忆中读取用户信息
    Returns:
    str: 用户信息
    """
    item = runtime.store.get(("users",), runtime.state["user_id"])
    return str(item.value) if item else "unknown"

agent = create_agent(
    model=model,
    tools=[save_user_info, get_user_info],
    store=store,
    system_prompt="用户提及个人信息时及时记录，用户询问个人信息时尝试用工具检索",
    state_schema=CustomState,
)

print("=" * 30, '-> 第一个会话（线程） <-', "=" * 30)
response1 = agent.invoke({
    "messages": [HumanMessage("你好，很高兴认识你，我是小花")],
    "user_id": "user-1"
})
for msg in response1["messages"]:
    msg.pretty_print()

print("=" * 30, '-> 第二个会话（线程） <-', "=" * 30)
response2 = agent.invoke({
    "messages": [HumanMessage("我是谁")],
    "user_id": "user-1"
})
for msg in response2["messages"]:
    msg.pretty_print()

```

```

===== -> 第一个会话（线程） <- =====
===== Human Message =====

你好，很高兴认识你，我是小花

===== Ai Message =====

你好，小花！很高兴认识你 😊 让我把你的名字记下来。
Tool Calls:
  save_user_info (call_00_3lb8itu3byQTibQoP6Er3151)
Call ID: call_00_3lb8itu3byQTibQoP6Er3151
Args:
  name: 小花

===== Tool Message =====
Name: save_user_info

saved

===== Ai Message =====

已经记住啦！以后我就叫你小花 🌸 有什么想聊的或者需要帮忙的，随时告诉我哦～
===== -> 第二个会话（线程） <- =====
===== Human Message =====

我是谁

===== Ai Message =====
Tool Calls:
  get_user_info (call_00_GWpSPoniT0648mSrEe4n7026)
Call ID: call_00_GWpSPoniT0648mSrEe4n7026
Args:

===== Tool Message =====
Name: get_user_info

{'name': '小花'}

===== Ai Message =====

你是小花呀！😊 我已经记住了你的名字。有什么我可以帮你的吗？

```

其中，CustomState扩展了Agent的标准状态。除了默认的messages（历史消息列表）之外，还额外增加了一个user_id字段。这样，Agent在运行过程中随时随地都能知道当前和它说话的用户ID是什么。

两次invoke没有通过config串联为起来，是两个独立的会话，但第二个会话可以访问第一个会话写入长期记忆的内容。

② 基于PostgresStore

```

In [48]: from typing import NotRequired
from langchain.agents import create_agent, AgentState
from langchain.tools import tool, ToolRuntime
from langgraph.store.postgres import PostgresStore
DB_URI = "postgresql://langchain_user:abcd1234@175.27.243.144:5432/langchain_db?sslmode=d

class CustomState(AgentState):
    user_id: NotRequired[str]

```



```

@tool(parse_docstring=True)
def save_user_info(name: str, runtime: ToolRuntime) -> str:
    """
    将用户信息保存在长期记忆中

    Args:
        name: 用户名

    Returns:
        str: 保存状态
    """
    runtime.store.put(("users",), runtime.state["user_id"], {"name": name})
    return "saved"

@tool(parse_docstring=True)
def get_user_info(runtime: ToolRuntime) -> str:
    """
    从长期记忆中读取用户信息

    Returns:
        str: 用户信息
    """
    item = runtime.store.get(("users",), runtime.state["user_id"])
    return str(item.value) if item else "unknown"

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    agent = create_agent(
        model=model,
        tools=[save_user_info, get_user_info],
        store=store,
        system_prompt="用户提及个人信息时及时记录，用户询问个人信息时尝试用工具检索",
        state_schema=CustomState,
    )

    print("=" * 30, '-> 第一个会话（线程） <-', "=" * 30)
    response1 = agent.invoke({
        "messages": "你好，很高兴认识你，我是小花",
        "user_id": "user-1"
    })
    for msg in response1["messages"]:
        msg.pretty_print()

    print("=" * 30, '-> 第二个会话（线程） <-', "=" * 30)
    response2 = agent.invoke({
        "messages": "我是谁",
        "user_id": "user-1"
    })
    for msg in response2["messages"]:
        msg.pretty_print()

```

```

===== -> 第一个会话（线程） <- =====
===== Human Message =====

你好，很高兴认识你，我是小花

===== Ai Message =====

你好，小花！很高兴认识你 🌸 让我把你的名字记下来，这样以后就能记得你了。
Tool Calls:
  save_user_info (call_00_zvv1EK59EzXPoKUTW6Gb7913)
Call ID: call_00_zvv1EK59EzXPoKUTW6Gb7913
Args:
  name: 小花

===== Tool Message =====
Name: save_user_info

saved

===== Ai Message =====

已经记住你啦，小花！以后我们聊天的时候我就知道是你的 😊 有什么我可以帮你的吗？
===== -> 第二个会话（线程） <- =====
===== Human Message =====

我是谁

===== Ai Message =====
Tool Calls:
  get_user_info (call_00_YTlqHdi0LKyERiclDeIJ8715)
Call ID: call_00_YTlqHdi0LKyERiclDeIJ8715
Args:

===== Tool Message =====
Name: get_user_info

{'name': '小花'}

===== Ai Message =====

你好，小花！你是我的用户，我们已经认识啦 😊

有什么我可以帮你的吗？

```

查看PostgreSQL数据表

查看表 输入 \dt

```

ubuntu@VM-0-12-ubuntu:~$ psql "postgresql://langchain_user:abcd1234@175.27.243.144:5432/langchain_db?sslmode=disable"
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

langchain_db=> \dt
               List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | checkpoint_blobs | table | langchain_user
public | checkpoint_migrations | table | langchain_user
public | checkpoint_writes | table | langchain_user
public | checkpoints      | table | langchain_user
public | store            | table | langchain_user
public | store_migrations | table | langchain_user
(6 rows)

```

新增两张Store相关的表store 和store_migrations。

3.3.2 在中间件中访问长期记忆

① Node-style hooks中访问

以before_model为例，其钩子函数签名如下

```
In [ ]: def before_model(self, state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None:
```

Runtime 定义如下

```
In [ ]: @dataclass(**_DC_KWARGS)
class Runtime(Generic[ContextT]):
    context: ContextT = field(default=None) # type: ignore[assignment]
    """Static context for the graph run, like `user_id`, `db_conn`, etc.

    Can also be thought of as 'run dependencies'."""

    store: BaseStore | None = field(default=None)
    """Store for the graph run, enabling persistence and memory."""

    stream_writer: StreamWriter = field(default=_no_op_stream_writer)
    """Function that writes to the custom stream."""

    previous: Any = field(default=None)
    """The previous return value for the given thread.

    Only available with the functional API when a checkpointer is provided.
    """
    ...
```

所以，我们可以通过runtime.store 在中间件中访问长期记忆。

② Wrap-style hooks中访问

1. wrap_model_call

钩子函数签名如下

```
In [ ]: def wrap_model_call(
    self,
    request: ModelRequest[ContextT],
    handler: Callable[[ModelRequest[ContextT]], ModelResponse[ResponseT]],
) -> ModelResponse[ResponseT] | AIMessage | ExtendedModelResponse[ResponseT]:
```

ModelRequest 定义如下

```
In [ ]: @dataclass(init=False)
class ModelRequest(Generic[ContextT]):
    """Model request information for the agent.

    Type Parameters:
        ContextT: The type of the runtime context. Defaults to `None` if not specified.
    """
```

```

model: BaseChatModel
messages: list[AnyMessage] # excluding system message
system_message: SystemMessage | None
tool_choice: Any | None
tools: list[BaseTool | dict[str, Any]]
response_format: ResponseFormat[Any] | None
state: AgentState[Any]
runtime: Runtime[ContextT]
model_settings: dict[str, Any] = field(default_factory=dict)
...

```

所以，可以通过request.runtime.store 访问长期记忆。

2. wrap_tool_call

钩子函数签名如下

```

In [ ]: def wrap_tool_call(
        self,
        request: ToolCallRequest,
        handler: Callable[[ToolCallRequest], ToolMessage | Command[Any]],
    ) -> ToolMessage | Command[Any]:

```

ToolCallRequest 定义如下

```

In [ ]: @dataclass
class ToolCallRequest:
    """Tool execution request passed to tool call interceptors.

    Attributes:
        tool_call: Tool call dict with name, args, and id from model output.
        tool: BaseTool instance to be invoked, or None if tool is not
            registered with the `ToolNode`. When tool is `None`, interceptors can
            handle the request without validation. If the interceptor calls `execute()`,
            validation will occur and raise an error for unregistered tools.
        state: Agent state (`dict`, `list`, or `BaseModel`).
        runtime: LangGraph runtime context (optional, `None` if outside graph).
    """

    tool_call: ToolCall
    tool: BaseTool | None
    state: Any
    runtime: ToolRuntime
    ...

```

ToolRuntime 定义如下

```

In [ ]: @dataclass
class ToolRuntime(_DirectlyInjectedToolArg, Generic[ContextT, StateT]):
    state: StateT
    context: ContextT
    config: RunnableConfig
    stream_writer: StreamWriter

```

```
tool_call_id: str | None
store: BaseStore | None
```

所以，可以通过`request.runtime.store` 访问长期记忆。

3.4 何时写入记忆

官方介绍了两种方式。

1. 在主流程里写（hot path）

也就是：用户发消息，AI 一边回答，一边决定要不要记下来。

优点：

- 立即生效
- 下一轮马上能用
- 用户可感知，透明

缺点：

- 增加延迟
- 逻辑变复杂

2. 在后台写（background）

就是先回答用户，记忆整理放到后台异步做。

优点：

- 主流程更快
- 记忆逻辑更独立
- 更适合批量整理

缺点：

- 不能立刻生效
- 要决定多久整理一次
- 触发时机不好选

工程上通常这么选：

- 用户偏好、账号资料：可热路径写
- 对话摘要、经验沉淀、行为分析：更适合后台写

4、课后阅读：Static runtime Context

静态运行时上下文(Static runtime Context)表示不可变的数据，如用户元数据、工具和传递给应用程序的数据库连接对象。通常在运行开始时通过 `invoke / stream` 的 `context` 参数传递，此类数据在运行期间不会更改。

4.1 中间件中访问

4.1.1 Node-style hooks

此类钩子都通过 `runtime.context` 访问上下文对象。

用户自定义 `ContextSchema` 用 `@dataclass` 修饰，在 `Agent` 创建时通过 `state_schema` 参数传递。

本例从长期记忆中查询用户额度，如果额度用尽则中断流程。

代码

```
In [50]: from dataclasses import dataclass
from typing import Any
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, AgentState, hook_config
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore
from dotenv import load_dotenv
from loguru import logger

load_dotenv()

store = InMemoryStore()
namespace = ("users", "credits")
key1 = 'Ada Lovelace'
value1 = {
    "tokens_credit_left": 5000,
    "user_level": 5
}
key2 = 'Blackwell'
value2 = {
    "tokens_credit_left": 2999,
    "user_level": 5
}
key3 = 'Ampere'
value3 = {
    "tokens_credit_left": 1000,
    "user_level": 5
}

store.put(namespace, key1, value1)
store.put(namespace, key2, value2)
store.put(namespace, key3, value3)

@dataclass
class UserContext:
```

```

username: str

class CheckCredit(AgentMiddleware):
    @hook_config(can_jump_to=["end"])
    def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        store = runtime.store
        context = runtime.context
        username = context.username
        credit = store.get(namespace, username)
        if not credit:
            return {
                "jump_to": "end",
                "messages": AIMessage("您尚未注册~")
            }

        if credit.value["tokens_credit_left"] < 3000:
            return {
                "jump_to": "end",
                "messages": AIMessage(f"{username}额度不足，请充值")
            }
        return None

    def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        store = runtime.store
        context = runtime.context

        username = context.username
        credit = store.get(namespace, username)
        usage_metadata = state["messages"][-1].usage_metadata
        token_usage = usage_metadata["input_tokens"] + usage_metadata["output_tokens"] * 6
        credit.value["tokens_credit_left"] -= token_usage
        # 更新长期记忆
        store.put(namespace, username, credit.value)
        return None

    def after_agent(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        store = runtime.store
        context = runtime.context
        username = context.username
        item = store.get(namespace, username)
        credit = item.value["tokens_credit_left"]
        logger.info(f"{username} 当前剩余额度：{credit}")
        return None

agent = create_agent(
    model="deepseek-v4-flash",
    middleware=[CheckCredit()],
    store=store,
    context_schema=UserContext
)
response1 = agent.invoke(
    {"messages": ["你好啊，你知道 Ada Lovelace 的贡献吗？"]},
    context=UserContext(username="Ada Lovelace")
)
print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)

```

```
for msg in response1["messages"]:
    msg.pretty_print()
response2 = agent.invoke(
    {"messages": ["你好啊，你知道 Blackwell 的贡献吗？"]},
    context=UserContext(username="Blackwell")
)
print("=" * 30, '-> Blackwell messages <-', '=' * 30)
for msg in response2["messages"]:
    msg.pretty_print()
```

2026-08-21 13:54:38.098	INFO	__main__:after_agent:78 - Ada Lovelace 当前剩余额度：-4372	▲
2026-08-21 13:54:38.106	INFO	__main__:after_agent:78 - Blackwell 当前剩余额度：2999	▼


```
===== -> Ada Lovelace messages <- =====
===
===== Human Message =====

你好啊，你知道 Ada Lovelace 的贡献吗？
===== Ai Message =====

你好！很高兴你问起她——是的，我非常了解 Ada Lovelace 的贡献，她可以说是计算机科学史上最举足轻重的先驱之一。

简而言之，她是**人类历史上第一位计算机程序员**。这比第一台真正的电脑问世，整整早了一个世纪。

她的核心贡献可以拆解为以下三个层面：

**1. 写下了世界上第一个“程序”**
1843年，Ada 在翻译一篇关于查尔斯·巴贝奇（Charles Babbage）设计的“分析机”（Analytical Engine）的意大利语论文时，加上了自己的大量注释。在这些注释中，她为分析机写下了**计算伯努利数**的详细操作步骤（即算法）。这套逻辑严密的“指令表”，被公认为世界上**第一个为机器设计的算法**。它包含了变量、循环、条件判断等现代编程的核心思想。

**2. 超前了时代的“通用计算”概念**
当时，巴贝奇本人只把分析机当作一台“高级计算器”。但Ada 敏锐地指出：**这台机器并不局限于处理数字，它可以处理任何能被符号化的东西**。她预言机器能够作曲、绘图、写作，甚至进行逻辑推理。她提出了“诗意科学”（Poetical Science）的概念，认为机器的潜力远比加法运算深远得多。这个“通用计算”的洞见，正是现代计算机（图灵完备）的本质。

**3. 开创了编程逻辑的先驱（但未曾被误解）**
她意识到“循环”和“子程序”的重要性，并建议分析机可以预先存储多个“公式卡”以便反复调用。她的注释在篇幅上是原论文的三倍多，但内容却比原作者深刻得多。

**关于她的一些小背景**：她的父亲是浪漫主义诗人拜伦勋爵，母亲则十分重视数学教育，特意培养她作为理科生。她虽然没有亲眼看到分析机被造出来（该机器因技术限制从未完全建成），但她的笔记在1953年（她去世后一百年）重见天日，震惊了整个计算机界。

为了纪念她，美国国防部在上世纪80年代开发的一种编程语言被命名为**Ada 语言**，而每年十月的第二个星期二被定为**Ada Lovelace Day**（阿达·洛夫莱斯日），用来纪念她对STEM（科学、技术、工程、数学）领域女性的杰出贡献。

可以说，如果没有她，计算机也许只是一台笨重的计数机器，而不会成为我们今天所知的、能处理一切事物的“通用大脑”。你对她的哪部分故事最感兴趣？我们可以继续深聊！
===== -> Blackwell messages <- =====
=
===== Human Message =====

你好啊，你知道 Blackwell 的贡献吗？
===== Ai Message =====

Blackwell额度不足，请充值
```

4.1.2 Wrap-style hooks

① wrap_model_call

本例通过 context 中记录的用户身份信息调整暴露给模型的工具集。

此处更改模型请求的方法是 transient request update ， 见上文短期记忆 2-4-2

```
In [52]: from dataclasses import dataclass
from typing import Callable
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelResponse, ModelRequest
from langchain.messages import AIMessage
from langchain.tools import tool
from langgraph.store.memory import InMemoryStore
from dotenv import load_dotenv
from loguru import logger

load_dotenv()
store = InMemoryStore()

namespace = ("users", "credits")
key1 = 'Ada Lovelace'
value1 = {
    "tokens_left": 3000, # 剩余大模型token额度
    "get_weather": "yes", # 是否认证天气查询服务
    "get_news": "no" # 是否认证网络爬虫服务, 用于查询新闻
}
key2 = 'Blackwell'
value2 = {
    "tokens_left": 2000, # 剩余大模型token额度
    "get_weather": "no", # 是否认证天气查询服务
    "get_news": "yes" # 是否认证网络爬虫服务, 用于查询新闻
}
key3 = 'Ampere'
value3 = {
    "tokens_left": 3000, # 剩余大模型token额度
    "get_weather": "yes", # 是否认证天气查询服务
    "get_news": "yes" # 是否认证网络爬虫服务, 用于查询新闻
}
store.put(namespace, key1, value1)
store.put(namespace, key2, value2)
store.put(namespace, key3, value3)

@tool
def get_weather(city: str):
    """
    查询指定城市当日天气

    Args:
        city: 城市名称
    """
    return f"{city} 今天天气不错"

@tool()
def get_news():
    """查询当日新闻"""
    return "美伊尚未达成停战协议"
```

```

@dataclass
class UserContext:
    username: str

class CheckCredit(AgentMiddleware):
    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse],
    ) -> ModelResponse:
        context = request.runtime.context
        store = request.runtime.store
        value = store.get(namespace, context.username).value
        tools = []
        for tool in request.tools:
            tool_name = tool.name
            if value[tool_name] == "yes":
                tools.append(tool)
            else:
                logger.warning(f"{context.username} 无权调用 {tool_name}")
                # 更改调用请求携带的工具集，仅本次生效
        request = request.override(tools=tools)

        return handler(request)

agent = create_agent(
    model="deepseek-v4-flash",
    middleware=[CheckCredit()],
    tools=[get_weather, get_news],
    store=store,
    context_schema=UserContext
)

response1 = agent.invoke(
    {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
    context=UserContext(username="Ada Lovelace")
)
print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
for msg in response1["messages"]:
    msg.pretty_print()

response2 = agent.invoke(
    {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
    context=UserContext(username="Blackwell")
)
print("=" * 30, '-> Blackwell messages <-', '=' * 30)
for msg in response2["messages"]:
    msg.pretty_print()

response3 = agent.invoke(
    {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
    context=UserContext(username="Ampere")
)
print("=" * 30, '-> Ampere messages <-', '=' * 30)
for msg in response3["messages"]:
    msg.pretty_print()

```

2026-08-21 14:02:56.858 | WARNING | __main__:wrap_model_call:71 - Ada Lovelace 无权调用 get_news
2026-08-21 14:02:58.166 | WARNING | __main__:wrap_model_call:71 - Ada Lovelace 无权调用 get_news
2026-08-21 14:02:59.859 | WARNING | __main__:wrap_model_call:71 - Blackwell 无权调用 get_weather

=====> Ada Lovelace messages <=====

=====
===== Human Message =====

你好啊，今天北京天气如何，今日有哪些新闻？

===== Ai Message =====

Tool Calls:

get_weather (call_00_X5LpqG7BpjNAtTCn47qd5273)

Call ID: call_00_X5LpqG7BpjNAtTCn47qd5273

Args:

city: 北京

===== Tool Message =====

Name: get_weather

北京 今天天气不错

===== Ai Message =====

你好！

关于你问的两件事：

🌤️ 北京天气

今天北京的天气不错哦，适合外出活动！☀️（具体气温、风力等细节建议以气象预报为准）

📰 今日新闻

抱歉，我目前没有查询实时新闻的工具，无法为你提供今日的新闻内容。建议你查看新闻类App或网站（如新华社、央视新闻、澎湃等）获取最新资讯。

如果有其他我能帮到的事情，比如查天气、规划行程等，随时告诉我！😊

2026-08-21 14:03:01.496 | WARNING | __main__:wrap_model_call:71 - Blackwell 无权调用 get_weather

===== -> Blackwell messages <- =====

=

===== Human Message =====

你好啊，今天北京天气如何，今日有哪些新闻？

===== Ai Message =====

Tool Calls:

get_news (call_00_fVe1uCWEWPZR1BjoGB2U1196)

Call ID: call_00_fVe1uCWEWPZR1BjoGB2U1196

Args:

===== Tool Message =====

Name: get_news

美伊尚未达成停战协议

===== Ai Message =====

你好！很高兴为你服务 😊

关于**北京天气**，很抱歉，我目前没有查询天气的功能，暂时无法为你提供今日北京的实时天气信息。建议你查看天气预报APP或网站获取准确数据。

关于**今日新闻**，我查询到的头条是：

> 📰 **美伊尚未达成停战协议**

如果你想了解这条新闻的更多细节，或者查询其他方面的信息，随时告诉我哦！

===== -> Ampere messages <- =====

===== Human Message =====

你好啊，今天北京天气如何，今日有哪些新闻？

===== Ai Message =====

Tool Calls:

get_weather (call_00_LDURMfSWkmNtfm6tJBCK3543)

Call ID: call_00_LDURMfSWkmNtfm6tJBCK3543

Args:

city: 北京

get_news (call_01_tVmzdoQkiTmlb65lqZvh7848)

Call ID: call_01_tVmzdoQkiTmlb65lqZvh7848

Args:

===== Tool Message =====

Name: get_weather

北京 今天天气不错

===== Tool Message =====

Name: get_news

美伊尚未达成停战协议

===== Ai Message =====

你好呀！已为你查询到以下信息：

🌞 北京今日天气

今天北京的天气不错哦，是个适合出门的日子。

📰 今日新闻

今天的头条新闻是：**美伊尚未达成停战协议**。

以上就是今天为你整理的天气和新闻，还有其他需要帮忙的吗？😊

② wrap_tool_call

在上述案例的基础上，我们补充额度校验，和长期记忆更新。

```
In [53]: from dataclasses import dataclass
from typing import Callable, Any
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelResponse, ModelRequest, ToolResponse
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool

from langgraph.store.memory import InMemoryStore
from langgraph.types import Command
from dotenv import load_dotenv
from loguru import logger
load_dotenv()

store = InMemoryStore()
namespace = ("users", "credits")
key1 = 'Ada Lovelace'
value1 = {
    "tokens_left": 3000, # 剩余大模型token额度
    "get_weather": "yes", # 是否认证天气查询服务
    "get_news": "no" # 是否认证网络爬虫服务，用于查询新闻
}
key2 = 'Blackwell'
value2 = {
    "tokens_left": 2000, # 剩余大模型token额度
    "get_weather": "no", # 是否认证天气查询服务
    "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
}
key3 = 'Ampere'
value3 = {
    "tokens_left": 3000, # 剩余大模型token额度
    "get_weather": "yes", # 是否认证天气查询服务
    "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
}
store.put(namespace, key1, value1)
store.put(namespace, key2, value2)
store.put(namespace, key3, value3)

# 工具额度消耗
CREDIT_MAP = {
    "get_weather": 10,
    "get_news": 20
}

@tool
def get_weather(city: str):
    """
    查询指定城市当日天气
    """
```

```

    Args:
        city: 城市名称
    """
    return f"{city} 今天天气不错"
@tool()
def get_news():
    """查询当日新闻"""
    return "美伊尚未达成停战协议"

@dataclass
class UserContext:
    username: str

class CheckCredit(AgentMiddleware):
    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse],
    ) -> ModelResponse:
        context = request.runtime.context
        store = request.runtime.store
        value = store.get(namespace, context.username).value
        tools = []
        for tool in request.tools:
            tool_name = tool.name
            if value[tool_name] == "yes":
                tools.append(tool)
            else:
                logger.warning(f"{context.username} 无权调用 {tool_name}")

        # 更改调用请求携带的工具集，仅本次生效
        request = request.override(tools=tools)

        return handler(request)

class ToolGuard(AgentMiddleware):
    def wrap_tool_call(
        self,
        request: ToolCallRequest,
        handler: Callable[[ToolCallRequest], ToolMessage | Command[Any]],
    ) -> ToolMessage | Command[Any]:
        store = request.runtime.store
        context = request.runtime.context
        username = context.username
        value = store.get(namespace, username).value
        tool_name = request.tool.name

        # 本次调用花费
        credits_cost = CREDIT_MAP.get(tool_name)

        # 剩余额度
        credits_left = value["tokens_left"]
        if credits_left < credits_cost:
            logger.warning(f"{username} 额度不足，{tool_name} 调用失败！")
        return Command(

```

```

        update={
            "messages": [
                ToolMessage(
                    content="额度不足，无法调用工具，请充值~",
                    tool_call_id=request.runtime.tool_call_id
                )
            ]
        }
    )

    # 更新剩余额度
    credits_left -= credits_cost
    value["tokens_left"] = credits_left
    store.put(namespace, username, value)
    logger.info(f"{username} 调用 {tool_name} 花费 {credits_cost}，剩余额度 {credits_left}")
    logger.info(f"{username} 状态已更新，当前状态 {store.get(namespace, username).value}")
    return handler(request)

agent = create_agent(
    model="deepseek-chat",
    middleware=[CheckCredit(), ToolGuard()],
    tools=[get_weather, get_news],
    store=store,
    context_schema=UserContext
)

response = agent.invoke(
    {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
    context=UserContext(username="Ada Lovelace")
)
print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
for msg in response["messages"]:
    msg.pretty_print()

```

```

2026-08-21 14:12:00.930 | WARNING | __main__:wrap_model_call:77 - Ada Lovelace 无权调用 get_news
2026-08-21 14:12:01.656 | INFO | __main__:wrap_tool_call:118 - Ada Lovelace 调用 get_weather 花费 10，剩余额度 2990
2026-08-21 14:12:01.664 | INFO | __main__:wrap_tool_call:119 - Ada Lovelace 状态已更新，当前状态 {'tokens_left': 2990, 'get_weather': 'yes', 'get_news': 'no'}
2026-08-21 14:12:01.668 | WARNING | __main__:wrap_model_call:77 - Ada Lovelace 无权调用 get_news

```



```

===== -> Ada Lovelace messages <- =====
===
===== Human Message =====

你好啊，今天北京天气如何，今日有哪些新闻？
===== Ai Message =====
Tool Calls:
  get_weather (call_00_EqgOkw64zyXfYhW8ND4l7057)
  Call ID: call_00_EqgOkw64zyXfYhW8ND4l7057
  Args:
    city: 北京
===== Tool Message =====
Name: get_weather

北京 今天天气不错
===== Ai Message =====

你好！

关于你的两个问题：

### 🌤️ 北京天气
根据查询，今天北京的天气**不错**，是个适合出门的好日子！

### 📰 今日新闻
很抱歉，我目前没有获取实时新闻的功能，无法为你提供今日的新闻资讯。建议你可以通过以下方式获取新闻：
- 打开新闻类App（如今日头条、腾讯新闻、网易新闻等）
- 访问各大新闻网站（如新华网、人民网、澎湃新闻等）
- 观看新闻频道或广播节目

如果你还有其他问题，比如查询其他城市的天气、或者需要其他帮助，随时告诉我哦！😊

```

4.1.3 特殊的便捷中间件dynamic_prompt

如前所述，@dynamic_prompt底层实现机制和通用钩子函数是统一的，对于动态提示词处理提供了更加便捷的入口。

本例通过静态上下文中携带的用户信息，动态更改系统提示词。

```

In [60]: from dataclasses import dataclass
          from langchain.agents import create_agent
          from langchain.agents.middleware import dynamic_prompt, ModelRequest
          from langgraph.store.memory import InMemoryStore
          from dotenv import load_dotenv
          from loguru import logger

          load_dotenv()

          # 准备历史记忆
          namespace = ("users", "preferences")
          key1 = "Ada Lovelace"
          value1 = {
            "chat_preferences": [
              "不喜欢啰嗦",

```

```

    "尽可能用最精简的文字解释清楚"
]
}
key2 = "Blackwell"
value2 = {
    "chat_preferences": [
        "喜欢长篇大论、引经据典",
        "不知道的东西要明确说'不知道'，不能胡编乱造"
    ]
}

store = InMemoryStore()

store.put(namespace, key1, value1)
store.put(namespace, key2, value2)

@dataclass
class UserContext:
    username: str

@dynamic_prompt
def personalized_prompt(request: ModelRequest) -> str:
    username = request.runtime.context.username
    store = request.runtime.store

    preferences = store.get(namespace, username).value["chat_preferences"]
    custom_prompt = "用户偏好\n" + '\n'.join(preferences)
    logger.info(f"{username} 自定义系统提示词\n{custom_prompt}")

    return custom_prompt

agent = create_agent(
    model="deepseek-chat",
    middleware=[personalized_prompt],
    store=store,
    context_schema=UserContext
)

response1 = agent.invoke(
    {"messages": ["为什么花儿这样红？"]},
    context=UserContext(username="Ada Lovelace")
)
print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
for msg in response1["messages"]:
    msg.pretty_print()

response2 = agent.invoke(
    {"messages": ["为什么花儿这样红？"]},
    context=UserContext(username="Blackwell")
)
print("=" * 30, '-> Blackwell messages <-', '=' * 30)
for msg in response2["messages"]:
    msg.pretty_print()

```

2026-08-21 14:20:28.575 | INFO | __main__:personalized_prompt:43 - Ada Lovelace 自定义系统提示词
用户偏好
不喜欢啰嗦
尽可能用最精简的文字解释清楚

2026-08-21 14:20:30.217 | INFO | __main__:personalized_prompt:43 - Blackwell 自定义系统提示词
用户偏好
喜欢长篇大论、引经据典
不知道的东西要明确说'不知道'，不能胡编乱造

```
===== -> Ada Lovelace messages <- =====
===
===== Human Message =====

为什么花儿这样红？
===== Ai Message =====

花红因花青素，遇酸变红，吸引传粉者。
===== -> Blackwell messages <- =====
=
===== Human Message =====

为什么花儿这样红？
===== Ai Message =====

这个问题问得好，既是一句熟悉的歌词，也是一个触及植物学、光学甚至演化论的深刻问题。让我从几个层面来拆解，尽量讲得透彻一些。

---

**首先，从光学和物理学的角度：**

我们看到的“红”，本质上是物体反射了特定波长的光。太阳光是由各种波长的光混合而成的（牛顿用三棱镜分解过）。当白光照射到花瓣上时，花瓣中的色素分子会吸收一部分波长的光，而反射另一部分。
- 花青素（尤其是呈现红色的花青素苷，如天竺葵素、矢车菊素等）在酸性细胞液中呈现红色，在碱性中偏蓝，在中性中偏紫。
- 类胡萝卜素（如番茄红素、胡萝卜素）则主要反射橙红色光。
所以，红花的红色，实际上是花瓣色素选择性吸收和反射的结果——它吸收了绿色、蓝色等波段，只把红色光“弹”回我们的眼睛。

---

**其次，从化学的角度：**

以常见的红花为例，其色素主要是**花青素**的衍生物。花青素本身是无色的（称为花青素苷元），但当它与糖结合成苷，并在细胞液pH、金属离子（如铝、铁）或分子间堆积等因素影响下，会形成稳定的红色显色基团。
例如，**天竺葵素**（Pelargonidin）在pH<3时呈红色，而矢车菊素（Cyanidin）在微酸性时也偏红。而细胞液的酸碱度又受到遗传和生理调控，所以同样是花青素，在不同植物里呈现的颜色千差万别。

---

**第三，从演化与生态学的角度（这可能是最迷人的部分）：**

为什么演化会“选择”红色？答案在于**传粉者**的感知偏好。

1. **蜜蜂的视觉**：蜜蜂能看到紫外光，但看不见纯红色（它们眼中的红色是暗色）。因此，红色花朵通常不依赖蜜蜂传粉。
2. **鸟类与蝴蝶**：许多鸟类（如蜂鸟）和某些鳞翅目昆虫对红色非常敏感。红色在绿叶背景中对比度极高，易于被发现。这种“广告色”能高效地吸引这些传粉者。
3. **异花授粉的智慧**：红色花朵往往在黄色或橙色花朵之后进化，它们通过独特的颜色来抢占传粉者注意力，避免与同类植物竞争。例如，在高海拔或干旱地区，红花比例更高，因为紫外线更强烈，红色色素能保护花瓣DNA免受辐射损伤。

---
```

****第四，从进化史的角度：****

最初的花（约1.4亿年前，被子植物早期）大多为绿色或白色，靠风或甲虫传粉。随着显花植物与昆虫、鸟类的协同演化，颜色逐渐多样化。****红色是相对“年轻”的颜色****——它依赖花青素的合成和pH调节，需要复杂的生化途径。所以，红花往往出现在更特化的类群中（如蔷薇科、石竹科、罂粟科等），是自然选择塑造的结果。

****最后，回到那句歌词与日常：****

“花儿为什么这样红”是经典歌曲《冰山上的来客》中的词句。它除了是美学感叹，也常被用作生物学教学的开场白。实际上，****红色是生存的呐喊，也是诱惑的艺术****——是植物用化学分子写成的诱导密码，是为传粉者点亮的路标。

如果还要再深一层，我可以聊：为什么有的花（如玫瑰）红色在衰老时会变蓝？这涉及细胞液pH变化和色素降解，甚至与果实成熟时的红变（香蕉、番茄）同源。但若你问的是最核心的答案，那就是：****化学决定的物理颜色，生态塑造的演化逻辑。****

补充一句，如果遇到我不知道或不确定的具体品种（比如某种特定兰花为何是红色），我会明确说不知道，而不去编造。但就“花儿为什么这样红”的普遍原理，上述解释覆盖了主流科学结论。你可以告诉我你更想深挖哪个方向，我再展开。

`dynamic_prompt` 是动态更新本次调用前的系统提示词，并不会更改消息列表，所以根据模型输出，可以发现系统提示词生效，但消息列表看不到相关提示词。

4.2 工具中访问

在上文介绍工具的自定义args_schema时我们介绍了两种方式：

- 基于pydantic模型
- 基于json_schema

实际上，Langchain底层在生成工具的schema时，如果参数类型注解是pydantic模型，也会被一并解析。

```
In [62]: from pydantic import BaseModel, Field
from typing import List
from rich import print as rprint

from langchain.tools import tool
from langgraph.prebuilt.tool_node import ToolRuntime
from langchain_core.utils.function_calling import convert_to_openai_tool

class UserInfo(BaseModel):
    username: str = Field(description="用户名")
    age: int = Field(description="年龄")
    hobbies: List[str] = Field(description="兴趣爱好")

@tool(parse_docstring=True)
```

```

def save_user_info(user_info: UserInfo, runtime: ToolRuntime) -> str:
    """
    保存用户信息

    Args:
        user_info: 用户信息对象
    """
    return "ok"

rprint(convert_to_openai_tool(save_user_info))
{
  'type': 'function',
  'function': {
    'name': 'save_user_info',
    'description': '保存用户信息',
    'parameters': {
      'properties': {
        'user_info': {
          'properties': {
            'username': {'description': '用户名', 'type': 'string'},
            'age': {'description': '年龄', 'type': 'integer'},
            'hobbies': {'description': '兴趣爱好', 'items': {'type': 'string'}}
          },
          'required': ['username', 'age', 'hobbies'],
          'type': 'object',
          'description': '用户信息对象'
        }
      },
      'required': ['user_info'],
      'type': 'object'
    }
  }
}

```

下面的案例中，我们通过工具读写长期记忆中保存的用户信息

```

In [63]: from pydantic import BaseModel, Field
from typing import List, Any
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage
from langgraph.prebuilt.tool_node import ToolRuntime
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import InMemorySaver
from loguru import logger
from dotenv import load_dotenv

load_dotenv()

@dataclass
class UserContext:
    user_id: str

```

```

store = InMemoryStore()
namespace = ("users", "user_info")

class UserInfo(BaseModel):
    username: str = Field(description="用户名", default="unknown")
    age: int = Field(description="年龄", default=0)
    # 这里相当于告诉模型，每次创建新实例时，调用list()创建一个新的list传进去
    hobbies: List[str] = Field(description="兴趣爱好", default_factory=list)
    # 这种写法是用可变对象做默认值，会有多个对象共享一个列表对象的风险
    # hobbies: List[str] = Field(description="兴趣爱好", default=[])

@tool(parse_docstring=True)
def read_user_info(runtime: ToolRuntime[UserContext, Any]) -> UserInfo | str:
    """
    读取用户信息

    Returns:
        UserInfo | str: 如果找到用户信息，返回 UserInfo 对象；否则返回空字符串。
    """
    user_id = runtime.context.user_id
    store = runtime.store
    item = store.get(namespace, user_id)
    if item:
        return UserInfo(**item.value)
    return ""

@tool(parse_docstring=True)
def write_user_info(user_info: UserInfo, runtime: ToolRuntime[UserContext, Any]) -> bool:
    """
    将用户信息写入长期记忆

    Args:
        user_info : 用户信息对象

    Returns:
        bool: 如果成功写入则返回True，否则返回False
    """
    user_id = runtime.context.user_id
    store = runtime.store
    try:
        logger.info(f"写入记忆, namespace: {namespace}, user_id: {user_id}, user_info: {user_info.model_dump()}")
        store.put(namespace, user_id, user_info.model_dump())
    except Exception as e:
        logger.error(e)
        return False
    return True

agent = create_agent(
    model = "deepseek-chat",
    tools = [read_user_info, write_user_info],
    store = store,
    checkpointer=InMemorySaver(),
    context_schema=UserContext
)

config1 = {"configurable": {"thread_id": "thread_1"}}

```

```

config2 = {"configurable": {"thread_id": "thread_2"}}
agent.invoke(
    {"messages": [
        SystemMessage("如果输入包含用户信息，抽取并调用工具记录。在记录之前先查找历史信息，和亲
        "如果用户提问设计个人信息，调用工具查找"),
        HumanMessage("你好，我是韩立，我喜欢修仙")
    ]},
    context = UserContext(user_id = 'user_1'),
    config = config1
)

thread_1_response = agent.invoke(
    {"messages": ["我今年两百岁了，是你们口中的“元婴老怪”，我喜欢跑步"]},
    context = UserContext(user_id = 'user_1'),
    config = config1
)
print("=" * 30, '-> thread_1 messages <-', '=' * 30)
for msg in thread_1_response["messages"]:
    msg.pretty_print()
thread_2_response = agent.invoke(
    {"messages": ["你还记得我吗？"]},
    context = UserContext(user_id = 'user_1'),
    config = config2
)
print("=" * 30, '-> thread_2 messages <-', '=' * 30)
for msg in thread_2_response["messages"]:
    msg.pretty_print()

```

```

2026-08-21 14:28:33.288 | INFO | __main__:write_user_info:59 - 写入记忆, namespace: ('users',
'user_info'), user_id: user_1, user_info: {'username': '韩立', 'age': 0, 'hobbies': ['修仙']}
2026-08-21 14:28:35.790 | INFO | __main__:write_user_info:59 - 写入记忆, namespace: ('users',
'user_info'), user_id: user_1, user_info: {'username': '韩立', 'age': 200, 'hobbies': ['修仙', '跑
步']}

```


===== -> thread_1 messages <- =====

===== **System Message** =====

如果输入包含用户信息，抽取并调用工具记录。在记录之前先查找历史信息，和新增信息合并后记录。
如果用户提问设计个人信息，调用工具查找

===== **Human Message** =====

你好，我是韩立，我喜欢修仙

===== **Ai Message** =====

我来帮您记录个人信息。首先让我查看一下已有的用户信息。

Tool Calls:

read_user_info (call_00_UTnZBOb3mCjkivsdNfbW6362)

Call ID: call_00_UTnZBOb3mCjkivsdNfbW6362

Args:

===== **Tool Message** =====

Name: read_user_info

===== **Ai Message** =====

目前系统中还没有历史信息。我来为您写入新的用户信息。

Tool Calls:

write_user_info (call_00_Ff3dzw2Wudrm6RAXLheF5370)

Call ID: call_00_Ff3dzw2Wudrm6RAXLheF5370

Args:

user_info: {'username': '韩立', 'hobbies': ['修仙']}

===== **Tool Message** =====

Name: write_user_info

true

===== **Ai Message** =====

您好，韩立！很高兴认识您。我已经将您的信息记录下来了：

- **用户名**：韩立

- **兴趣爱好**：修仙

请问还有什么我可以帮您吗？

===== **Human Message** =====

我今年两百岁了，是你们口中的“元婴老怪”，我喜欢跑步

===== **Ai Message** =====

我来为您更新年龄和新增的兴趣爱好，先查找一下历史记录确认信息。

Tool Calls:

read_user_info (call_00_rlyu47n6X9QxcaFPGJSB8819)

Call ID: call_00_rlyu47n6X9QxcaFPGJSB8819

Args:

===== **Tool Message** =====

Name: read_user_info

username='韩立' age=0 hobbies=['修仙']

===== **Ai Message** =====

我之前记录的信息确认无误。现在我来为您更新年龄和新增兴趣爱好。

Tool Calls:

write_user_info (call_00_XMjXxTvyQzEmMaB9fS3i4499)

Call ID: call_00_XMjXxTvyQzEmMaB9fS3i4499

Args:

user_info: {'username': '韩立', 'age': 200, 'hobbies': ['修仙', '跑步']}

===== Tool Message =====

Name: write_user_info

true

===== Ai Message =====

已经为您更新好个人信息了，韩立道友：

- **用户名**：韩立

- **年龄**：200岁（元婴老怪，失敬失敬！）

- **兴趣爱好**：修仙、跑步

没想到您这位元婴老祖还喜欢跑步，真是道法自然，动静结合啊！还有什么需要我帮忙的吗？

===== -> thread_2 messages <- =====

===== Human Message =====

你还记得我吗？

===== Ai Message =====

让我先查看一下你的信息。

Tool Calls:

read_user_info (call_00_fX7jvUPTK8p7b3SpKvgR1440)

Call ID: call_00_fX7jvUPTK8p7b3SpKvgR1440

Args:

===== Tool Message =====

Name: read_user_info

username='韩立' age=200 hobbies=['修仙', '跑步']

===== Ai Message =====

当然记得你！让我确认一下：

- **用户名**：韩立

- **年龄**：200岁

- **兴趣爱好**：修仙、跑步

看来你是一位跨入修仙之道的朋友呢，还保持着跑步锻炼的习惯，很不错！😊

有什么我可以帮你的吗？

分析

1. 前两次invoke通过相同的thread_id串联为一个会话，共享短期记忆
2. context是运行时静态配置，每次invoke互相独立
3. 长期记忆可以在任意位置访问，全局共享同一份信息
4. tool参数列表中的ToolRuntime是Langchain / Langgraph运行时注入的，在工具schema中没有体现
5. ToolRuntime的默认泛型是[None, Any]，第一个是Context的泛型，第二个是State对象的泛型。如果不显式指定上下文对象的类型，则Langchain/Langgraph底

层会认为上下文对象为None，invoke调用时传递了context就会抛出警告，所以这里将UserContext作为ToolRuntime的第一个泛型传递。